STUDY BUDDY STEM AI: AN AI-POWERED PERSONALIZED, ADAPTIVE AND MULTIMODAL STEM LEARNING ECOSYSTEM FOR SRI LANKAN STUDENTS

INDIVIDUAL THESIS

W.B.M.D.K. Basnayaka

IT22272768

Emotion-Aware Adaptive Assessment

R26-IT-009

B.Sc. (Hons) Degree in Information Technology

Specializing in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: W.B.M.D.K. Basnayaka

Student ID: IT22272768

Signature: _____  _____

Date: ____10/11/2026_____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Aruna Ishara Gamage		
Co-supervisor: Jeewaka Perera		

ABSTRACT

Assessments taken by Sri Lankan GCE O/L students in STEM subjects usually judge a learner only by whether an answer is correct, give every learner the same questions and feedback, and leave revision planning to the learner. This individual component of the Study Buddy STEM AI platform addresses these gaps with an emotion-aware adaptive assessment built around four features. First, a facial-expression signal from a custom-trained YOLOv8s detector is monitored during each question, and the hint is shown immediately when frustration is detected, before a wrong answer is submitted. Second, when a learner retakes the assessment, questions from concepts answered wrongly in the previous session are prioritised, with at least one weak-concept question guaranteed at each of the three difficulty levels. Third, every session produces a personalised study plan: dated action cards for each weak concept and a seven-day timetable whose intensity follows the learner's score, which the learner can customise, save and reset. Fourth, a T5-based generator turns a teacher's PDF into multiple-choice questions grouped into three levels without manual authoring. These features are supported by a Random Forest learning-state classifier and a literature-grounded cognitive load score that combine correctness, response time, answer changes and facial expression. The author built the component with React, FastAPI and MongoDB. Automated tests confirmed that retakes contain one weak-concept question per level and exactly nine questions, and that the assessment continues when the machine-learning service is unavailable. The classifier reached 97.9% cross-validated accuracy on a 140-record researcher-prepared dataset, compared with 87.9% for a rule-based baseline, and the detector achieved 0.905 mAP@0.5. The question generator produced correctly leveled questions, although distractor quality needs improvement. The component is a working prototype; a user study with real learners is the next step to measure learning gains.

Keywords: adaptive assessment, personalised study plan, affective computing, automatic question generation, cognitive load

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my supervisor, Aruna Ishara Gamage, and co-supervisor, Jeewaka Perera, for their guidance, feedback and encouragement throughout this research. Their advice on the research direction and on the evaluation of the adaptive assessment component shaped the quality of this work.

I also thank the members of my research group, Elwitigala S.A.I., Manodya K.M.B. and Dilshan W.A., for their collaboration in building the complete learning platform and for integrating their components with mine. I am grateful to the academic and non-academic staff of the Faculty of Computing at the Sri Lanka Institute of Information Technology for providing the facilities and resources used in this project.

Finally, I thank my family and friends for their patience and continuous support during my studies.

TABEL OF CONTENTS

1. INTRODUCTION 1
1.1 Overall Project Context 1
1.2 Individual Component Context 2
1.3 Problem Statement 2
1.4 Research Gap 3
1.5 Aim and Objectives 4
1.6 Research Questions or Hypotheses 5
1.7 Scope Limitations and Assumptions 6
1.8 Individual Contribution 7
1.9 Thesis Structure 8
2 LITERATURE REVIEW AND RELATED WORK 9
2.1 Review Method and Scope 9
2.2 Domain and Theoretical Foundations 9
2.2.1 Cognitive load theory 9
2.2.2 Self-regulated learning and answer revision 10
2.2.3 Affect in learning 10
2.2.4 Zone of proximal development and mastery 10
2.2.5 Formative feedback 11
2.3 Related Research 11
2.3.1 Adaptive testing and learner modelling 11
2.3.2 Behavioural indicators of learner state 11
2.3.3 Affect detection in educational systems 12
2.3.4 Real-time object detection 12
2.3.5 Automatic question generation 12

2.3.6 Gamification 13
2.4 Existing Systems Products and Techniques 13
2.5 Gap Synthesis and Conceptual Framework 14
3.RESEARCH METHODOLOGY 15
3.1 Research Design 15
3.2 Data Participants or Study Materials 15
3.2.1 Learning-state dataset 15
3.2.2 Facial-expression dataset 16
3.2.3 Question bank and generation materials 17
3.2.4 Participants 17
3.3 Experimental or Study Procedure 17
3.4 Measures and Evaluation Metrics 18
3.5 Data Analysis 19
3.6 Ethics Privacy Security and Responsible AI 19
3.7 Reproducibility and Research Artefacts 20
4.PRODUCT AND SOLUTION DESIGN 21
4.1 Requirements 21
4.1.1 Functional requirements 21
4.1.2 Non-functional requirements 22
4.2 Component Architecture within the Complete Solution 22
4.3 Design Alternatives and Decisions 24
4.4 Detailed Design 24
4.4.1 Assessment flow 24
4.4.2 Learning-state classifier 25
4.4.3 Cognitive load score 25
4.4.4 Difficulty adaptation policy 26

4.4.5 Support and feedback policy 26
4.4.6 Expression detection 27
4.4.7 Data model 27
4.4.8 Reporting, study plan and next-session personalisation 28
4.4.9 PDF question generation 28
4.5 Standards and Professional Practice 29
5.IMPLEMENTATION INTEGRATION AND TESTING 30
5.1 Development Environment and Technology Stack 30
5.2 Component Implementation 30
5.2.1 Frontend assessment engine 30
5.2.2 Backend API 35
5.2.3 ML service 36
5.2.4 Teacher dashboard 36
5.2.5 Personalised study plan and timetable 38
5.3 Integration with the Other Components 41
5.4 Verification and Software Testing 42
5.5 Deployment and Operation 45
6.RESULTS AND VALIDATION 46
6.1 Research Results 46
6.1.1 Learning-state prediction (RQ1) 46
6.1.2 Feature contribution (RQ2) 48
6.1.3 Adaptive-policy behaviour (RQ4) 49
6.2 Product and Technical Results 50
6.2.1 Facial-expression detector (RQ3) 50
6.2.2 Other technical results 51
6.3 User or Stakeholder Validation 51

6.4 Integration Results 52
6.5 Summary of Findings 52
7.DISCUSSION 53
7.1 Interpretation of Findings 53
7.2 Comparison with Literature Baselines and Existing Solutions 54
7.3 Research Contribution 55
7.4 Product Contribution 55
7.5 Limitations and Threats to Validity 55
7.6 Implications 56
8.COMMERCIALISATION AND SUSTAINABILITY 57
8.1 Value Proposition and Target Users 57
8.2 Market and Competitive Position 57
8.3 Business or Sustainability Model 57
8.4 Scalability Intellectual Property and Compliance 58
8.5 Contribution to the Sustainable Development Goals 58
9.CONCLUSION AND FUTURE DIRECTIONS 59
9.1 Conclusion 59
9.2 Achievement of Objectives 60
9.3 Individual Research and Product Contributions 60
9.4 Future Directions 61
REFERENCES 62
GLOSSARY 66
APPENDICES 67
Appendix A Research Instruments or Extended Results 67
A.1 Planned user-validation protocol 67
A.2 System Usability Scale items 67

A.3 Sample records from the learning-state dataset 68
A.4 Application screenshots 68
Appendix B Academic Integrity and AI Use Disclosure 71

LIST OF FIGURES

Figure 2.1: Conceptual framework of the adaptive assessment component 14
Figure 4.1: Architecture of the adaptive assessment component as merged into Study Buddy STEM AI 23
Figure 5.1: Cognitive load, adaptive-level and support-mode functions (quizLogic.js, abridged) 31
Figure 5.2: Lesson-completed page with the weak concepts prioritised in the next session 33
Figure 5.3: Assessment page during a Level 1 question, with the face-tracking panel 34
Figure 5.4: Feedback after a correct answer, with the predicted learning state 35
Figure 5.5: Teacher dashboard with the PDF question generator 37
Figure 5.6: Student performance overview in the teacher dashboard 37
Figure 5.7: Assessment report page with badges, progress summary and level-wise performance 40
Figure 5.8: Generated weekly study timetable with customisation options 41
Figure 6.1: Cross-validated accuracy (with standard deviation) and macro F1 of the proposed model and baselines 46
Figure 6.2: Confusion matrix of the Random Forest (out-of-fold predictions; counts and row percentages) 47
Figure 6.3: Random Forest feature importance (mean decrease in impurity) 48
Figure 6.4: Mean cognitive load and adaptive decisions per simulated learner profile 49
Figure 6.5: YOLOv8s training curves over 60 epochs: training and validation losses, precision, recall and mAP (blue: per-epoch values; orange: smoothed) 50
Figure A.1: Login and registration page 68
Figure A.2: Progress over time across five sessions of one learner 69
Figure A.3: Engagement timeline, cognitive effort chart and learning-state distribution in the report 70

LIST OF TABLES

Table 1.1: Components of the Study Buddy STEM AI project 1
Table 2.1: Comparison of the proposed component with existing approaches..... 13
Table 3.1: Features and label of the learning-state dataset 16
Table 3.2: Mean feature values per learning state in the training dataset 16
Table 4.1: Functional requirements..... 21
Table 4.2: Key design decisions 24
Table 4.3: Rule-based fallback for learning-state prediction (evaluated in order) 25
Table 4.4: MongoDB collections 27
Table 5.1: Technology stack 30
Table 5.2: Main backend endpoints 35
Table 5.3: Timetable intensity levels 39
Table 5.4: Changes made when merging the component into Study Buddy STEM AI 42
Table 5.5: Executed test cases and observed results 43
Table 6.1: Learning-state prediction, stratified five-fold cross-validation (n = 140). 46
Table 6.2: Per-class results of the Random Forest (out-of-fold predictions)..... 47
Table 6.3: Leave-one-feature-out ablation of the Random Forest 48
Table 6.4: Simulated behaviour of the adaptive policy (500 sessions per profile) 49
Table 6.5: Expression detector validation metrics (from the trained checkpoint) 50
Table 6.6: Summary of findings by research question..... 52
Table 9.1: Achievement of the research objectives 60
Table A.1: Example records from training_data.csv 68
Table B.1: Use of generative AI and automated tools 71

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
AQG	Automatic Question Generation
CAT	Computerised Adaptive Testing
CDN	Content Delivery Network
CLT	Cognitive Load Theory
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
CV	Cross-Validation
DSR	Design Science Research
JWT	JSON Web Token
LLM	Large Language Model
mAP	mean Average Precision
NASA-TLX	NASA Task Load Index
NIE	National Institute of Education
OCR	Optical Character Recognition
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SDG	Sustainable Development Goal
SPA	Single-Page Application
STEM	Science, Technology, Engineering and Mathematics
T5	Text-to-Text Transfer Transformer
ViT	Vision Transformer
YOLO	You Only Look Once

1. INTRODUCTION

1.1 Overall Project Context

Digital learning platforms are now a normal part of school education, and artificial intelligence makes it possible to personalise them: content can be adapted to the learner, tutoring can be offered at any time, and feedback can be generated automatically. The research project to which this thesis belongs, Study Buddy STEM AI, is an AI-powered, personalised, adaptive and multimodal STEM learning ecosystem for Sri Lankan General Certificate of Education Ordinary Level (GCE O/L) students. It integrates four independently developed but interconnected components, listed in Table 1.1, which share a knowledge layer of curriculum resources and NIE textbooks and a common layer of retrieval, API, machine-learning and storage services.

Table 1.1: Components of the Study Buddy STEM AI project

Component	Group member	Main techniques
RAG-Based Personalized STEM Storytelling	Elwitigala S.A.I.	Random Forest persona classification, syllabus-anchored RAG with ChromaDB, LLM narrative generation
Multilingual Conversational AI	Manodya K.M.B.	Whisper speech-to-text, conversation memory, RAG retrieval, LLM tutoring responses
Emotion-Aware Adaptive Assessment (this thesis)	Basnayaka W.B.M.D.K.	YOLOv8s facial-expression detection, Random Forest learning-state prediction, cognitive load scoring, T5 question generation
Handwritten Content Analysis and Synthesis (Knowledge Maps)	Dilshan W.A.	Region-aware OCR, local vision LLM (Gemma 4 via Ollama), hybrid retrieval, mind maps and flashcards

Together the components form one learning workflow: a learner first studies a concept through personalised storytelling, asks follow-up questions through the conversational tutor, demonstrates understanding through the adaptive assessment, and finally uploads handwritten notes for verification and synthesis. The platform is built as a single FastAPI application in which each component is a separate package mounted under its own API prefix. The component described in this thesis is the assessment step of this workflow: an emotion-aware adaptive assessment based on behavioural–affective fusion, mounted at /api/adaptive-quiz. The reference lesson

used to develop and test it is a Biology lesson, “The Human Heart”, which covers the four chambers, the major blood vessels, the valves and the two circulatory loops.

1.2 Individual Component Context

Assessment is the point at which a learning platform finds out what the learner actually understood, and it is also the point at which the platform can intervene. Most quiz engines, however, record only the selected option and whether it was correct. They ignore how the learner arrived at the answer: how long they took, whether they changed their mind, and whether they looked confused or frustrated while working. Research on cognitive load [1], [2] and on affect during learning [3], [4] shows that these process signals carry information about the learner's state that a single correct or incorrect flag does not.

The individual component therefore treats every question as a small behavioural observation and responds to it. While the learner works, the browser measures response time and counts answer changes, and the webcam is sampled at a fixed interval to estimate a facial-expression label. If the learner looks frustrated, the hint for the question is shown immediately, before an answer is submitted. When the learner submits a wrong answer, the component marks the correct answer next to the learner's choice and shows a short theory explanation, so that the mistake becomes a learning moment rather than only a lost mark. At the same time it predicts a learning state, calculates a cognitive load score and decides how the next difficulty level should be delivered. At the end of the session it produces a report and a personalised study plan, and stores a weak-area profile that the next session uses to prioritise questions from concepts the learner answered wrongly. A teacher-facing panel turns PDF learning material into new leveled questions so that the question bank can grow without manual authoring.

1.3 Problem Statement

Existing assessment tools used after STEM lessons give the same sequence of questions and the same feedback to every learner, and they judge understanding only by final correctness. As a result, they cannot distinguish a learner who answers correctly with confidence from one who guesses after long hesitation, and they

cannot detect when a learner is overloaded or frustrated until the learner has already failed. When an answer is wrong, many tools show only a mark, without the correct answer or an explanation, so the mistake does not become a learning opportunity. When the learner takes the assessment again, the questions are not focused on the concepts that were answered wrongly before, and the learner is left to plan revision alone, without a personalised plan that can be adjusted to their own time. Finally, question difficulty is not adjusted to keep the learner in a productive level of challenge, and teachers lack a fast way to produce leveled questions from their own material.

The problem addressed by this component can be stated as follows: **how can a post-lesson STEM assessment use observable behavioural and affective signals, captured unobtrusively during question answering, to estimate the learner's learning state and cognitive load in real time, and use those estimates to give timely support (an immediate hint when frustration is detected, and the correct answer with a theory explanation after a mistake), to adapt question difficulty, to prioritise previously missed concepts when the assessment is retaken, and to produce a personalised, customisable study plan, while allowing teachers to generate leveled questions from their own documents?**

1.4 Research Gap

Section 2 reviews the related work in detail; the gap is summarised here. Computerised adaptive testing [5] and knowledge tracing [6], [7] adapt question selection, but they model only response correctness and do not use response behaviour or affect. Affect-aware tutoring systems [8], [3] detect emotions, but they usually rely on specialised sensors or laboratory setups, and they mostly respond with motivational messages after a problem has occurred rather than offering task-specific help, such as the hint for the current question, at the moment frustration appears. Work on cognitive load [1], [2] provides well-established theory, but practical systems rarely turn that theory into an explicit, explainable per-question score that directly drives difficulty changes.

Research on formative feedback [9] shows that learners benefit most when a mistake is followed immediately by the correct answer and an explanation, yet many quiz tools still return only a mark. Adaptive platforms track mastery, but few use the concepts a learner answered wrongly in one assessment to prioritise the questions of the next attempt in a short post-lesson quiz, and fewer still turn the results into a personalised revision plan with dated tasks and a weekly timetable that the learner can adjust and save. Automatic question generation [10] is usually studied separately from the adaptive assessment that would consume the generated questions. In addition, most personalised-learning systems adapt only to ability level and are not designed for the local GCE O/L context.

The gap addressed in this thesis is therefore the absence of an integrated, low-cost, browser-based assessment component that (a) fuses correctness, response time, answer revisions and webcam-based facial expression; (b) converts them into a machine-learned learning state and a transparent cognitive load score grounded in learning theory; (c) uses these to give immediate, task-specific support, showing the hint as soon as frustration is detected and the correct answer with a theory explanation after every mistake, and to adapt difficulty in both directions; (d) prioritises previously missed concepts when the assessment is retaken and produces a personalised, customisable study plan; and (e) extends its question bank automatically from teacher documents.

1.5 Aim and Objectives

Aim. To design, implement and evaluate an emotion-aware, multi-level adaptive assessment that uses behavioural and facial-expression signals to support learners and adapt difficulty during post-lesson STEM assessment.

Specific objectives.

1. To review the literature on adaptive assessment, cognitive load, affective computing and question generation, and derive the features and thresholds used by the component.
2. To build and evaluate a model that classifies each response into one of four learning states from behavioural and facial-expression features.

3. To integrate a real-time webcam expression detector that shows the hint immediately when frustration is detected.
4. To define a cognitive load score and a three-level adaptation policy, and to show the correct answer with a theory explanation after every wrong answer.
5. To generate leveled multiple-choice questions from teacher PDF documents.
6. To produce session reports and a customisable study plan, and to prioritise previously missed concepts when the assessment is retaken.
7. To verify the component and evaluate its models and adaptive behaviour through tests, offline experiments and simulation.

1.6 Research Questions or Hypotheses

The work is guided by the following research questions:

- **RQ1.** Can a lightweight machine-learning model predict a learner's learning state from per-question behavioural and affective features more accurately than a hand-written rule set and a trivial baseline?
- **RQ2.** Which features contribute most to learning-state prediction, and does facial expression add predictive value beyond correctness, time and answer changes?
- **RQ3.** Can a pre-trained object-detection model provide a usable real-time facial-expression signal from an ordinary laptop webcam in a browser-based assessment?
- **RQ4.** Does the proposed cognitive load score and adaptation policy respond to different learner profiles in the intended way, accelerating fluent learners and giving overloaded learners a scaffolded support mode?
- **RQ5.** Can the component deliver timely, task-specific support (an immediate hint when frustration is detected and the correct answer with a theory explanation after a mistake), prioritise previously missed concepts when the assessment is retaken, and produce a customisable study plan reliably within a single integrated system?

RQ1 to RQ4 address the models and the adaptation policy, and RQ5 addresses the learner-facing support features. The corresponding working hypothesis for RQ1 is

that a Random Forest classifier will exceed both the rule-based fallback and the majority-class baseline in cross-validated accuracy and macro F1 score.

1.7 Scope Limitations and Assumptions

The component covers the assessment that follows a completed lesson. Its scope includes the adaptive quiz flow, learning-state prediction, facial-expression detection, cognitive load scoring, difficulty adaptation, hints, theory explanations, reports, study plans, the teacher dashboard and PDF-based question generation. The lesson itself and any other group components are outside the scope of this thesis except where they integrate with the assessment.

The following limitations and assumptions apply:

- The learning-state model is trained on a 140-record labelled dataset prepared by the researcher from expected learner behaviour, not on data logged from real learners. Results on this dataset show feasibility, not real-world accuracy.
- No formal study with students was carried out within the project timeframe; evaluation is based on offline model metrics, software testing and simulation. A user-study protocol is provided for future work.
- The facial-expression model detects three valence classes (negative, neutral, positive) which are mapped to frustrated, neutral and happy. The confused and surprised categories used in the theory are therefore not produced by the deployed detector.
- The learner is assumed to use a device with a webcam and a modern browser and to give consent for camera use. If the camera is unavailable, the system continues with a neutral expression label.
- Question difficulty levels are assigned by the question author or, for generated questions, by a sentence-length heuristic rather than by psychometric calibration.

1.8 Individual Contribution

The author was responsible for the complete adaptive assessment component, which includes:

- the React assessment interface, including the three-level quiz flow, timer, answer-change tracking, webcam capture hook, gamification bar, hint and theory panels;
- the cognitive load model and the bidirectional difficulty-adaptation and proactive-support logic;
- the FastAPI backend for authentication, assessments, responses, reports, study plans, learning-state prediction proxying and question generation, and the MongoDB data model;
- the Random Forest learning-state model, its training dataset and the rule-based fallback;
- the integration of the YOLOv8 facial-expression detector, including class mapping and a smile-aware decision rule;
- the T5-based PDF question generator and the teacher dashboard;
- the report, badge and study-plan generation, and the evaluation reported in Chapter 6.

The novelty of the component lies in five design choices:

- **Bidirectional, dual-signal adaptation.** The learner is accelerated only when correctness and low cognitive load agree, and scaffolded only when incorrectness and high load agree, so a single lucky guess under strain or a single slip under low strain does not change the level, unlike simple right/wrong adaptive engines.
- **Multimodal behavioural–affective fusion.** Facial expression, response time and answer-revision count are combined into one composite score rather than used as separate, disconnected signals, so the score reflects genuine struggle rather than one noisy signal.
- **Proactive rather than reactive intervention.** The hint system monitors the live expression stream during the attempt and surfaces support the moment frustration is detected, instead of waiting for a wrong answer.

- **A custom-trained detector.** The expression detector is a YOLOv8s model trained by the researcher on a custom-labelled dataset rather than a general-purpose third-party emotion API, which keeps detection tunable to the classroom-quiz context and keeps images in-house.
- **Literature-grounded thresholds.** Every scoring threshold (the 45-second overload zone, the three-revision uncertainty signal and the frustration-versus-confusion weighting) is tied to published cognitive-load and educational-psychology research.

1.9 Thesis Structure

Chapter 2 reviews the theoretical foundations and related work and synthesises the research gap into a conceptual framework. Chapter 3 describes the research methodology, data, evaluation metrics and ethical considerations. Chapter 4 presents the requirements and the design of the component. Chapter 5 describes the implementation, integration and testing. Chapter 6 reports the results of the model evaluation, the expression detector, the adaptive-policy simulation and verification. Chapter 7 discusses the findings, limitations and implications. Chapter 8 considers commercialisation and sustainability, and Chapter 9 concludes the thesis and outlines future work.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review Method and Scope

The review was organised around the four capabilities that the component combines: adaptive assessment and learner modelling, cognitive load and its behavioural indicators, affect detection in learning, and automatic question generation. Sources were identified from IEEE Xplore, the ACM Digital Library, Google Scholar and the reference lists of key review papers, using search terms such as “adaptive testing”, “knowledge tracing”, “cognitive load response time”, “affect-aware tutoring”, “facial expression recognition students” and “automatic question generation”. Priority was given to peer-reviewed journal articles, highly cited conference papers, and the original descriptions of the algorithms and models used in the implementation. To keep the review current, only sources published in 2018 or later were used; classical theories on which the component's thresholds are based, such as cognitive load theory, self-regulated learning and the zone of proximal development, are cited through recent reviews and meta-analyses that summarise and update them.

2.2 Domain and Theoretical Foundations

2.2.1 Cognitive load theory

Cognitive Load Theory (CLT) holds that working memory has limited capacity, and that learning is impaired when the total load imposed by a task exceeds that capacity [1]. Load is commonly divided into intrinsic load, caused by the complexity of the material; extraneous load, caused by poor presentation; and germane load, the effort that contributes to building schemas. Paas and van Merriënboer [2] summarise how mental effort can be measured and combined with performance, and how working-memory load can be managed when learners work on complex tasks. Self-report scales such as NASA-TLX [11] are widely used but interrupt the learner, which makes them unsuitable for per-question measurement. Behavioural proxies such as response time are therefore attractive because they can be captured continuously and without interruption.

2.2.2 Self-regulated learning and answer revision

Winne [12] describes self-regulated learning as a process in which learners monitor their own understanding at several levels of processing and adjust their actions accordingly. Changing an answer is an observable metacognitive monitoring event: a single change often reflects healthy self-correction, but repeated changes indicate uncertainty about the concept. This motivates the use of the number of answer changes as a feature and the choice of three or more changes as a signal of genuine uncertainty in this component.

2.2.3 Affect in learning

Emotions influence attention, motivation and the use of learning strategies. A meta-analysis of achievement emotions [13] found that positive activating emotions such as enjoyment are associated with better academic performance, whereas negative emotions such as anxiety and boredom are associated with poorer performance. An integrative review of confusion in learning [3] concludes that confusion can be productive when the learner is able to resolve it, whereas unresolved confusion tends to turn into frustration and disengagement. A review of epistemic-emotion management in digital learning environments [4] similarly argues that systems should detect confusion and frustration early and intervene before these states persist and lead learners to give up. Together these findings justify (a) treating frustration as a stronger warning signal than confusion and (b) intervening early, before a negative state persists.

2.2.4 Zone of proximal development and mastery

Vygotsky's Zone of Proximal Development (ZPD) [14] describes the range of tasks a learner can complete with support but not yet alone. Tasks below this zone produce little learning, and tasks above it produce failure. A meta-analysis of experimental studies [15] found that individual tutoring produces large and consistent learning gains, which supports individualised progression in which learners move forward only when they show mastery. Flow theory [16] similarly argues that engagement is highest when challenge matches skill. These ideas underpin the bidirectional difficulty policy in Section 4.4: raise difficulty on fluent mastery, lower it on overload, and otherwise stay in the productive-struggle zone.

2.2.5 Formative feedback

A meta-analysis of educational feedback research [9] concludes that feedback is most effective when it carries high information, telling learners not only whether an answer was right but also why and how to improve, and that simple right or wrong messages are much less effective. This supports showing a short theory explanation after every incorrect answer and showing hints selectively when the learner shows signs of struggle.

2.3 Related Research

2.3.1 Adaptive testing and learner modelling

Computerised Adaptive Testing (CAT) selects each next item based on an estimate of learner ability, typically using Item Response Theory [5]. CAT is efficient for measurement but requires items calibrated on large samples, and it uses only correctness. Knowledge tracing models estimate the probability that a learner has mastered each skill; as recent surveys describe [6], [7], Bayesian Knowledge Tracing uses a hidden Markov model and Deep Knowledge Tracing uses recurrent neural networks over long interaction sequences. These approaches work well with large logs, but they need many interactions per learner, they are hard to explain to teachers, and they also ignore behavioural and affective context. Reviews and meta-analyses of intelligent tutoring systems [17], [18] report positive effects on learning, which shows the value of adaptivity, but most of the reported systems adapt mainly on performance data such as correctness.

2.3.2 Behavioural indicators of learner state

Response time and answer revisions are among the cheapest learner-state signals available. Long response times on familiar material have been associated with high cognitive load [2], while very short times can indicate either fluency or guessing. Combining time with correctness separates these cases: fast and correct suggests fluency, slow and correct suggests effortful success, and fast and incorrect suggests guessing or a misconception. The present component uses exactly this combination as input to its learning-state model.

2.3.3 Affect detection in educational systems

Affect-aware learning systems have often combined several input modalities to recognise emotion; for example, Standen et al. [8] evaluated an adaptive learning system that used multimodal affect recognition to adapt learning activities for learners with intellectual disabilities. A review of engagement detection in online learning [19] shows that computer-vision methods based on facial expressions are among the most widely used approaches for recognising student engagement. These studies show that facial expression is informative, but many rely on dedicated hardware or laboratory conditions. Deep facial expression recognition has advanced through large public datasets such as FER2013 and modern deep architectures [20], and most of these datasets still use the basic-emotion categories. Basic emotions, however, do not map neatly to learning-centred states such as confusion, and the same facial movement can express different emotions in different contexts [21], so educational systems often regroup them, as this component does.

2.3.4 Real-time object detection

The You Only Look Once (YOLO) family [22] frames detection as a single regression problem and runs in real time. The Ultralytics YOLOv8 implementation [23] provides small pre-trained models that can be fine-tuned on custom classes and run on a CPU, which makes it practical to detect a face and classify its expression in one pass without the separate face-detection stage of classical two-stage pipelines [24].

2.3.5 Automatic question generation

Kurdi et al. [10] reviewed automatic question generation (AQG) for education and found that most systems generate factual questions from text, that distractor quality is a recurring weakness, and that evaluation is often limited. Earlier systems used rule-based syntactic transformation and ranking, whereas recent systems use neural sequence-to-sequence models [25]. Transformer models such as T5 [26] cast question generation as a text-to-text task in which the model receives an answer span and its context and produces a question. Small T5 models fine-tuned for answer-aware question generation are fast enough to run on a CPU, which is important for a teacher tool.

2.3.6 Gamification

Gamification applies game design elements such as points, progress bars and badges in non-game contexts [27]. A meta-analysis of gamified learning [28] found small positive effects on cognitive, motivational and behavioural learning outcomes, although the effects depend on context and on the learner. The component uses experience points, an energy bar and badges as low-stakes motivational support rather than as the core assessment mechanism.

2.4 Existing Systems Products and Techniques

Table 2.1 compares the component with representative categories of existing tools. Commercial quiz platforms such as Kahoot and Quizizz focus on engagement and teacher-paced quizzes. Adaptive learning platforms such as Khan Academy's mastery system and ALEKS adapt content based on correctness. Affect-aware research tutors detect emotion but are rarely deployed as lightweight web tools. None of the surveyed categories combines behavioural signals, webcam affect, an explicit cognitive load score, bidirectional level adaptation, correct-answer feedback with theory explanations, a customisable study plan and automatic question generation in one component.

Table 2.1: Comparison of the proposed component with existing approaches

Capability	Quiz platforms (e.g. Kahoot)	Mastery platforms (e.g. Khan Academy, ALEKS)	Affect-aware research tutors	Proposed component
Adapts difficulty	No	Yes (correctness)	Partly	Yes (correctness + load)
Uses response time and revisions	Score bonus only	Limited	Sometimes	Yes
Uses facial expression	No	No	Yes (often special sensors)	Yes (ordinary webcam)
Explicit cognitive load score	No	No	Rarely	Yes (0–100)
Proactive hints	No	Hints on request	Yes	Yes
Correct answer with theory explanation after a mistake	Correct answer only	Yes (worked explanations)	Often	Yes (every incorrect answer)
Customisable study plan	No	Partly (system-set learning path)	Rarely	Yes (learner-customised)
Generates questions from teacher PDFs	No	No	No	Yes (T5)
Cross-session weak-area targeting	No	Yes	Varies	Yes

2.5 Gap Synthesis and Conceptual Framework

The review shows three things. First, theory on cognitive load, affect, self-regulation and ZPD consistently suggests that the process of answering carries information about the learner that correctness alone does not. Second, existing adaptive systems mostly ignore that information, while affect-aware systems tend to be heavy, sensor-based and focused on motivational messages rather than on assessment decisions. Third, AQG is mature enough to support teachers but is rarely connected to an adaptive assessment loop.

The conceptual framework of this component, shown in Figure 2.1, links observable signals to inferred states and then to actions. Observable signals (correctness, response time, answer changes, quiz level and facial expression) feed two inference paths: a data-driven learning-state classifier and a theory-driven cognitive load score. The inferred states drive three kinds of action: in-question support (proactive hints), post-answer feedback (the correct answer with a theory explanation, and hints), and difficulty adaptation (scale up, hold or scale down into support mode). Session outcomes are stored as weak areas that personalise the next session and the customisable study plan, closing the loop.

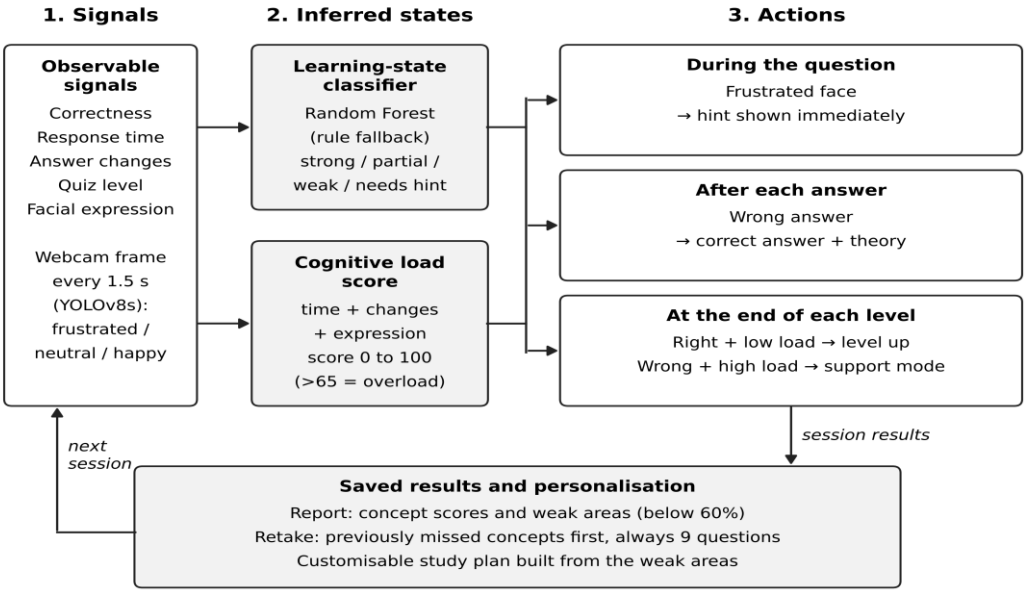


Figure 2.1: Conceptual framework of the adaptive assessment component

3.RESEARCH METHODOLOGY

3.1 Research Design

The research follows a Design Science Research (DSR) approach, in which knowledge is produced by building and evaluating an artefact that addresses a practical problem. The artefact is the adaptive assessment component. The work proceeded through iterative cycles of requirement identification from literature, design, implementation and evaluation. Within the evaluation stage, a quantitative experimental design was used for the machine-learning parts (comparison against baselines with cross-validation, and feature ablation), a simulation design was used to examine the behaviour of the rule-based adaptive policy, and software verification was used to check functional correctness and integration.

Development followed an incremental agile process. The component was first built with a Node.js and Express backend and was later migrated to a Python FastAPI backend so that the API and the machine-learning code share one language and so that the webcam model could be served by the same process. Each iteration added a capability, such as learning-state prediction, webcam expression detection, cognitive load and adaptive levels, PDF question generation, and reports and study plans, and was tested before the next iteration began.

3.2 Data Participants or Study Materials

3.2.1 Learning-state dataset

The learning-state classifier was trained on a labelled dataset of 140 records stored in training_data.csv. Each record describes one question attempt with five input features and one label (Table 3.1). Because no logged learner data was available at the time of model development, the records were prepared by the researcher to represent expected behaviour patterns for each learning state, informed by the literature in Chapter 2. The class distribution is approximately balanced: 38 partial understanding, 38 weak understanding, 33 needs hint and 31 strong understanding. Facial expressions in the dataset are neutral (47), confused (33), happy (28), frustrated (25) and surprised (7). Four records are exact duplicates.

Table 3.1: Features and label of the learning-state dataset

Field	Type	Values	Meaning
correctness	Integer	0 or 1	Whether the submitted answer was correct
responseTime	Float	Seconds (3–65 in data)	Time from question display to submission
answerChanges	Integer	0–8 in data	Number of times the selected option was changed
quizLevel	Integer	1, 2, 3	Basic, concept understanding, advanced
faceExpression	Category	neutral, happy, confused, frustrated, surprised	Dominant expression at submission (encoded 0–4)
label	Category	strong, partial, weak understanding; needs hint	Target learning state

Table 3.2 summarises how the features differ across the labels. Correct answers are labelled strong or partial understanding, and incorrect answers are labelled weak understanding or needs hint. Within each pair, response time and answer changes separate the classes.

Table 3.2: Mean feature values per learning state in the training dataset

Learning state	n	Correct	Mean time (s)	Time range (s)	Mean changes
Strong understanding	31	100%	8.7	3–16	0.2
Partial understanding	38	100%	24.3	13–40	2.3
Weak understanding	38	0%	16.4	6–29	0.9
Needs hint	33	0%	42.6	28–65	4.7

3.2.2 Facial-expression dataset

The facial-expression detector is a YOLOv8s model trained by the researcher on a custom-labelled facial-expression detection dataset (project name “Confusion-level-1”) with three valence classes: negative, neutral and positive. Each image was labelled by drawing a bounding box around the face and assigning one of the three classes. The model checkpoint (best.pt, 22.5 MB) records the training configuration: the model was trained in Google Colab with Ultralytics 8.2.103 for 60 epochs at an image size of 640 and a batch size of 16, starting from COCO-pretrained yolov8s weights, and the dataset was loaded from an exported dataset folder named Confusion-level-1. The validation metrics stored in the checkpoint are used in Chapter 6.

3.2.3 Question bank and generation materials

The reference question bank contains nine expert-written questions for the Human Heart lesson, three per level, covering eight concept tags (heart function, heart structure, blood vessels, heart chambers, circulation, cardiac physiology, heart sounds and cardiac pathology). Each question has four options, a correct answer, a hint and a short theory explanation. For the question generator, a one-page test document (test_accuracy_sample.pdf) containing 20 factual sentences from several STEM and general-knowledge topics was used to exercise text extraction, answer-candidate extraction, distractor selection and level assignment.

3.2.4 Participants

No human participants were recruited for this phase of the research. Section 6.3 describes the user-validation protocol that is planned for the next phase.

3.3 Experimental or Study Procedure

Five evaluation procedures were carried out.

Procedure 1: learning-state model comparison (RQ1). The proposed Random Forest was compared with four baselines on the same dataset: a majority-class classifier, the rule-based fallback implemented in the backend, a depth-2 decision tree and a standardised logistic regression. All learned models were evaluated with stratified five-fold cross-validation using a fixed random seed (42) so that every model saw the same folds. The rule-based fallback has no parameters and was applied directly to all records.

Procedure 2: feature contribution (RQ2). Feature importance was taken from the mean decrease in impurity of the Random Forest trained on all data. In addition, a leave-one-feature-out ablation was run: the Random Forest was retrained five times, each time without one feature, and evaluated with the same cross-validation.

Procedure 3: expression detector (RQ3). The validation precision, recall, mAP@0.5 and mAP@0.5:0.95, and the per-epoch training curves stored in the checkpoint, were extracted. The integrated inference function was run on test frames to confirm that the model loads, that the class mapping works and to measure CPU inference time per frame.

Procedure 4: adaptive-policy simulation (RQ4). Three synthetic learner profiles were defined: a high performer (90% base accuracy, 6–14 s per question, 0–1 changes, mostly happy or neutral), an average learner (65% accuracy, 12–35 s, 0–2 changes, mixed expressions) and a struggling learner (35% accuracy, 25–60 s, 1–5 changes, often frustrated). Accuracy decreased and time increased with level. Each profile completed 500 simulated nine-question sessions. For every question the cognitive load score was calculated, and at the end of each level the adaptive policy decided whether to scale up, hold or scale down. The trained Random Forest was also applied to the first 40 sessions of each profile to observe the predicted learning-state mix.

Procedure 5: functional verification of the support features (RQ5). The learner-facing features were verified with automated tests (Section 5.4). Retake prioritisation and session length were tested through the adaptive-question endpoint and a reducer simulation of 2,000 sessions; the study-plan endpoints were tested for saving, reloading, updating, resetting and per-student isolation; and the feedback rules were tested as pure functions to confirm that a wrong answer always triggers the correct answer with a theory explanation and that a frustrated expression triggers the hint. These tests establish that the features behave as designed; their effect on learning is left to the planned user study.

3.4 Measures and Evaluation Metrics

The following metrics were used:

- **Accuracy**: the proportion of correctly classified records.
- **Macro-averaged F1 score**: the unweighted mean of per-class F1 scores, which treats every learning state as equally important.
- **Per-class precision and recall** and the **confusion matrix**, computed from out-of-fold cross-validated predictions.
- **Feature importance** (mean decrease in impurity) and **ablation accuracy drop**.
- **Detection metrics** for the expression model: precision, recall, mAP@0.5 and mAP@0.5:0.95.

- **Latency**: mean time per prediction for the Random Forest and per frame for the YOLO model on CPU.
- **Policy measures** in simulation: mean cognitive load, percentage of questions above the overload threshold (65) the percentage of level-end decisions that scaled up, and the percentage of level transitions made in support mode.

3.5 Data Analysis

All analysis was performed in Python 3.13 with pandas, NumPy and scikit-learn 1.9 [29]. Cross-validation followed common practice for small datasets [30]: stratified folds preserve the class balance in each fold, and the mean and standard deviation over folds are reported. Because the dataset is small, results are interpreted as evidence about the separability of the labelled patterns rather than as estimates of real-world accuracy. Model checkpoint metrics were read with PyTorch and Ultralytics. Figures were produced with Matplotlib. The scripts used for every reported number are stored with the project (Section 3.7).

3.6 Ethics Privacy Security and Responsible AI

Webcam-based emotion detection raises clear privacy and ethical concerns, particularly because the target users may be minors. The design addresses these concerns as follows:

- **Data minimisation.** Webcam frames are sampled at 320×240 pixels, sent to the backend for inference and discarded immediately. No image is stored; only the resulting expression label is saved with each response.
- **Consent and control.** The camera is activated only on the assessment page through the browser's permission prompt, and the assessment continues with a neutral label if permission is refused or the camera fails.
- **Security.** Student endpoints require a JSON Web Token (JWT), passwords are hashed with bcrypt, login and registration are rate-limited, every student-specific endpoint checks that the token owner matches the requested student identifier, and teacher endpoints require a shared teacher key compared in constant time. These measures follow common OWASP guidance [31].

- **Responsible use of AI outputs.** Learning-state predictions and expression labels are used only to decide on supportive actions such as hints, correct-answer explanations and difficulty changes. They are never used to penalise a learner's score, and the report presents them as indicators rather than judgements.
- **Regulation.** The handling of personal data should comply with the Sri Lanka Personal Data Protection Act No. 9 of 2022 [32] and, for international deployment, the General Data Protection Regulation and its guidance on valid consent [33], including informed parental consent for minors.
- **Bias.** Facial-expression models can perform differently across skin tones, ages and lighting conditions. This risk is acknowledged as a limitation in Section 7.5, and the system design ensures that a wrong expression label can at most trigger an unnecessary hint or skip one; it cannot change the score.

Before any user study, ethical clearance will be obtained from the relevant SLIIT committee, together with informed consent from participants and, for minors, from parents or guardians.

3.7 Reproducibility and Research Artefacts

The research artefacts are organised in a single repository with three top-level folders: frontend (React application), backend (FastAPI API, MongoDB models, seed script and the YOLO checkpoint in backend/model/best.pt) and ml-service (FastAPI ML service, training data, training script and the serialised Random Forest in models/learning_state_model.pkl). The model can be reproduced with python modules/train.py, which prints the cross-validation accuracy, feature importances, held-out classification report and confusion matrix. The seed script seed_data.py recreates the reference lesson and its nine questions. All random processes use fixed seeds (42 for model training and cross-validation, 7 for the simulation). Dependency versions are pinned in backend/requirements.txt, and the frontend dependencies are locked in package-lock.json.

4.PRODUCT AND SOLUTION DESIGN

4.1 Requirements

Requirements were derived from the research objectives, the literature review and the needs of the two user groups: learners who take the assessment and teachers who review results and add questions.

4.1.1 Functional requirements

The functional requirements are listed in Table 4.1, together with the research objective that each one supports.

Table 4.1: Functional requirements

ID	Requirement	Objective
FR1	Learners can register, log in and stay authenticated across pages.	–
FR2	After a lesson, the system presents nine questions in three levels (basic, concept, advanced).	4
FR3	The system records response time and answer changes for every question.	2
FR4	The system samples the webcam during each question and obtains an expression label.	3
FR5	On submission, the system predicts the learning state and falls back to rules if the ML service is unavailable.	2
FR6	The system computes a cognitive load score and decides the next level at the end of each level.	4
FR7	The system shows a hint immediately when frustration is detected (and other proactive hints during a question), and shows the correct answer with a theory explanation after every incorrect answer.	4
FR8	The system generates a report with level scores, concept scores, weak areas, expression frequency, learning-state distribution, hints and time.	6
FR9	The system generates a study plan and weekly timetable, which the learner can customise and save.	6
FR10	The next session guarantees at least one question per level from a previous weak concept.	6
FR11	Teachers can view all reports and generate leveled questions from a PDF.	5
FR12	Learners can generate a private practice quiz from their own PDF without affecting graded data.	5

4.1.2 Non-functional requirements

- **Performance.** Expression inference should complete well within the 1.5-second sampling interval, and learning-state prediction should not noticeably delay feedback.
- **Availability.** The assessment must remain usable if the ML service or the webcam is unavailable.
- **Security and privacy.** Authentication, ownership checks, rate limiting and no storage of images (Section 3.6).
- **Usability and accessibility.** Clear level and progress indicators, simple controls and on-screen feedback that do not distract from the question.
- **Maintainability.** Prediction, quiz logic and HTTP layers are kept in separate modules so that models can be replaced without changing the API contract.

4.2 Component Architecture within the Complete Solution

The component was first developed as a stand-alone three-service application (React frontend on port 5173, FastAPI backend on port 3001 and ML service on port 8000) and was then merged into the Study Buddy STEM AI platform. In the merged build (Figure 4.1) it runs as two processes plus a cloud database. The main Study Buddy STEM AI FastAPI application on port 8000 hosts all /api/adaptive-quiz routes, serves the built React single-page application under /adaptive-quiz, runs the YOLOv8 expression detector in-process and owns the MongoDB Atlas connection. A small ML micro-service on port 8001 hosts the Random Forest classifier and the T5 question generator, which have heavier dependencies and slower start-up. The browser talks only to the main application, and the main application calls the ML service over HTTP with httpx.

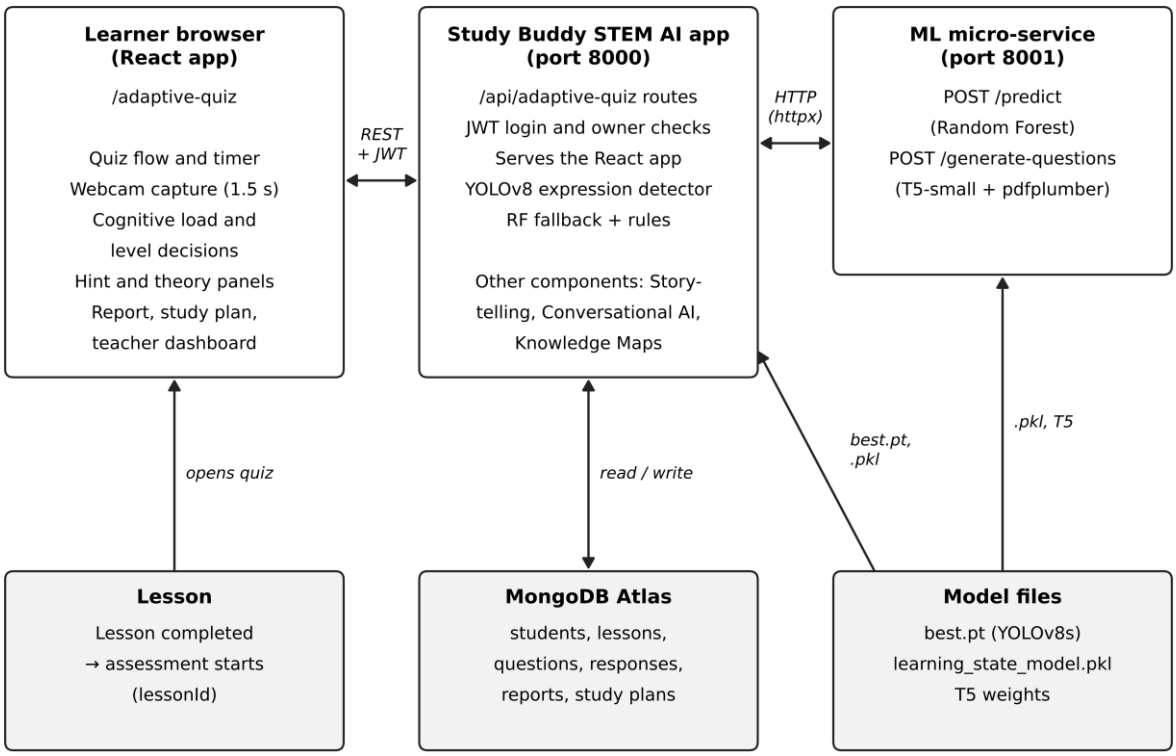


Figure 4.1: Architecture of the adaptive assessment component as merged into Study Buddy STEM AI

The component is isolated from the rest of the platform. Its router package (backend/components/adaptive_quiz/) is mounted like the other components, and its MongoDB connection is opened inside the shared application lifespan in a guarded block: if MONGO_URI is missing or Atlas is unreachable, only the quiz routes return HTTP 503, while the storytelling, conversational AI and Knowledge Maps components keep working. A status endpoint reports the readiness of the database, the expression model and the learning-state model. The component integrates with the learning flow at the lesson boundary. When a learner completes a lesson, the platform navigates to the lesson-completed page with the lesson identifier. The assessment component loads the lesson record (including its arModuleId) and the questions for that lesson, so that any lesson created by the platform can be assessed without code changes.

4.3 Design Alternatives and Decisions

Table 4.2 summarises the main design decisions, the alternatives that were considered and the reason for each choice.

Table 4.2: Key design decisions

Decision	Alternatives considered	Choice and rationale
Learning-state model	Rules only; logistic regression; neural network; knowledge tracing	Random Forest: handles mixed numeric and categorical features, works on small data, gives class probabilities and feature importances, and needs no feature scaling. Rules kept as fallback.
Load estimation	Learned regression; self-report (NASA-TLX); physiological sensors	Explicit weighted score from literature: transparent, needs no training data and does not interrupt the learner.
Expression detector	Separate face detector plus CNN classifier; ViT classifier; cloud API	Fine-tuned YOLOv8s: one-pass face localisation and classification, runs on CPU, no third-party data transfer. A ViT classifier remains in the ML service as an alternative.
Backend framework	Node.js/Express (initial version); Django	FastAPI: same language as the ML code, asynchronous I/O, automatic request validation and OpenAPI documentation.
Database	Relational database	MongoDB: report and study-plan structures are nested and evolve; documents map directly to the JSON used by the frontend.
Where adaptation runs	Server side	Client side for immediate feedback; all inputs are saved server side so reports can be recomputed.
Question generation	Template rules; large hosted LLM	T5-small answer-aware generation: runs locally on CPU with int8 quantisation, no per-request cost or data sharing.

4.4 Detailed Design

4.4.1 Assessment flow

A session begins when the learner starts the assessment for a lesson. The frontend calls the adaptive questions endpoint, which returns three questions for each level, nine in total. If the learner has a previous report for the lesson, the endpoint guarantees one question per level from a previous weak concept and fills the remaining slots randomly from the other questions of that level; otherwise all questions are drawn at random. A unique session identifier is created from the student identifier, the lesson identifier and a timestamp. The learner then answers questions level by level. Figure 2.1 shows how the signals from each question lead to support, feedback and adaptation decisions.

4.4.2 Learning-state classifier

The classifier receives the feature vector [correctness, responseTime, answerChanges, quizLevel, faceExpression], where the expression is encoded as neutral 0, happy 1, confused 2, frustrated 3 and surprised 4. It is a Random Forest [34] of 200 trees with a maximum depth of 8 and balanced class weights. The service returns the predicted state, its probability and the probabilities of all four states. Prediction uses a three-tier path: the backend first calls the ML service; if it is unreachable, it uses an in-process copy of the same Random Forest; and only if that also fails does it apply the rule-based fallback in Table 4.3.

Table 4.3: Rule-based fallback for learning-state prediction (evaluated in order)

Condition	**Learning state**
Correct, response time < 15 s, expression not confused or frustrated	strong_understanding
Incorrect and expression confused or frustrated	needs_hint
Incorrect, or response time > 30 s, or more than 2 answer changes	weak_understanding
Otherwise	partial_understanding

4.4.3 Cognitive load score

The cognitive load score L (0–100) is the sum of three components, clipped to the range 0–100:

$$L = clip(T(t) + min(10 \times c, 30) + E(e), 0, 100)$$

where t is the response time in seconds, c is the number of answer changes and e is the expression. The time component T(t) is 40 if $t > 45$ s (overload zone [1]), 20 if $25 < t \leq 45$ s (elevated, productive effort [2]) and 0 otherwise. The uncertainty component adds 10 points per revision up to 30, reflecting repeated metacognitive monitoring [12]. The affect component E(e) is +30 for frustrated, +20 for confused and −10 for happy, reflecting the stronger link of frustration to disengagement [3], [4] and the load-reducing effect of positive affect [13].

4.4.4 Difficulty adaptation policy

At the last question of each level, the policy decides the next level from the correctness of that answer and its load score:

- **Scale up** if the answer is correct and L < 35: mastery with low effort, the accelerated-learning zone [16].
- **Scale down** if the answer is incorrect and L > 65: the task exceeds the learner's ZPD and scaffolding is needed [14]. Because the levels are always visited in order, this decision puts the next level (level 2 or level 3) into support mode; it can therefore follow an overloaded level 1 as well as an overloaded level 2.
- **Stay** otherwise: the learner is in productive struggle.

Every session has a fixed length of nine questions: each level is visited exactly once, in the order 1, 2, 3, so that every learner answers the same number of questions and session results stay comparable. The policy decision therefore changes how the next level is delivered rather than which questions are asked. After a scale-down decision, the next level runs in support mode: the hint of every question is opened from the start and the learner is told that the level will be taken step by step. After a hold or scale-up decision, the next level runs normally, with hints appearing only through the proactive and post-answer rules. Every decision is stored with the response as adaptiveLevel for analysis. When the third level is complete, the session ends and the report is generated.

4.4.5 Support and feedback policy

Support is given at two moments. During a question, a proactive message with the hint is shown if the learner looks frustrated for 10 seconds, looks confused for 15 seconds, has changed the answer three or more times, or has spent 40 seconds on the question. A frustrated expression also opens the hint immediately. After submission, the hint is shown if the answer is incorrect and the learner needs a hint (predicted needs_hint, confused or frustrated expression, or response time of at least 30 seconds), and for every incorrect answer the correct option is marked next to the learner's choice and a theory explanation box shows the correct answer with a short explanation. Correct answers increase experience points and energy; incorrect answers reduce energy.

4.4.6 Expression detection

The frontend captures a frame at a fixed interval (1.5 s in the stand-alone build and about 0.8 s in the merged build) and sends it as a base64 JPEG to the backend. The YOLOv8 detector returns boxes with classes negative, neutral and positive. For each class the score is the maximum of confidence multiplied by the square root of the normalised box area, so that the main (largest) face dominates. Because the model tends to label subtle smiles as neutral, the positive class is preferred when its score is within 12% of the best score, or when the best class is neutral and the positive score is at least 52% of it. The raw class is then mapped to the system label (negative to frustrated, neutral to neutral, positive to happy). The mapping can be extended through an environment variable without code changes. To keep the server responsive, inference is single-flight: it runs on a dedicated one-worker thread pool, and a frame that arrives while an inference is running is answered immediately with the last result, so a burst of frames cannot block prediction or database requests. The merged build also uses a smaller inference size (256 pixels instead of 640) for speed.

4.4.7 Data model

Table 4.4 lists the MongoDB collections. Each StudentResponse stores the behavioural features, expression, predicted learning state and hint usage, linked to a session. The AssessmentReport is computed from the responses of a session and is the source for progress tracking and adaptive question selection.

Table 4.4: MongoDB collections

Collection	Main fields
students	studentId, name, email, password hash, gradeLevel, completedLessons
lessons	lessonId, title, subject, gradeLevel, arModuleId, description, estimatedDuration
questions	lessonId, questionId, quizLevel, conceptTag, questionText, options, correctAnswer, hint, shortTheoryExplanation, source
studentresponses	studentId, lessonId, questionId, quizLevel, conceptTag, selectedAnswer, correctness, responseTime, answerChanges, detectedExpression, learningState, hintUsed, sessionId
assessmentreports	sessionId, totalScore, levelScores, conceptPerformance, weakAreas, mostCommonExpression, expressionFrequency, learningStateDistribution, hintUsageCount, totalTime, progressSummary
studyplans	studentId, lessonId, schedule, intensity, weeklyMinutes, weekRangeLabel

4.4.8 Reporting, study plan and next-session personalisation

The report computes the total score, the score per level and per concept, and marks every concept below 60% as a weak area. It also records the most common expression, the expression frequency, the learning-state distribution, the number of hints used and the total time. The study plan creates one specific, dated task for every weak concept (for example raising a 33% concept to 53% within seven days), adds behavioural advice based on load, persistence, dominant expression and hint dependence, and recommends a next action. A weekly timetable distributes study sessions across the week; the learner can edit and save it. Badges are awarded for achievements such as a perfect score, improvement over the previous session and hint-free completion. The progress endpoint compares sessions and reports score changes, improved concepts and persistent weak areas. Section 5.2.5 describes the study plan and timetable in detail.

4.4.9 PDF question generation

The generator extracts text with pdfplumber, skipping front-matter pages numbered with Roman numerals. It splits the text into sentences of at least six words and extracts answer candidates: capitalised phrases, quoted phrases and numbers with up to two following words (with trailing stop words removed). For each candidate it builds the input “answer: <answer> context: <sentence>” and runs the valhalla/t5-small-qg-prepend model with greedy decoding in batches of 32, using int8 dynamic quantisation on CPU. Three distractors are drawn from other capitalised phrases in the document, excluding phrases that share the first word with the answer or contain it, and preferring phrases of similar length. Questions are sorted by sentence length and split evenly into levels 1, 2 and 3. The sentence becomes the theory explanation and its first 160 characters the hint.

4.5 Standards and Professional Practice

- **API design.** RESTful endpoints with consistent JSON responses ({success, message, data}) and OpenAPI documentation generated by FastAPI.
- **Security.** JWT bearer tokens (RFC 7519), bcrypt password hashing, rate limiting, CORS restrictions and object-level authorisation checks, following OWASP Top 10 guidance [31].
- **Data protection.** Privacy-by-design and data minimisation as required by data-protection legislation [32], [33].
- **Code quality.** ESLint for the frontend, typed Pydantic models for request validation, separation of concerns between HTTP routes, services and pure logic modules, and version control with Git.
- **Open-source licences.** Ultralytics YOLOv8 is AGPL-3.0, which affects commercial distribution (Section 8.4).

5.IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development Environment and Technology Stack

Table 5.1 lists the technologies used in each layer of the component.

Table 5.1: Technology stack

Layer	Technologies
Frontend	React 19, Vite 8, React Router 7, Axios, Recharts, MediaDevices (getUserMedia)
Backend API	Python, FastAPI 0.115, Uvicorn, Beanie 1.27 and Motor 3.6 (MongoDB), PyJWT, bcrypt, SlowAPI, HTTPX
Expression detection	Ultralytics YOLOv8 (yolov8s fine-tuned), OpenCV, NumPy, PyTorch
ML service	FastAPI, scikit-learn, pandas, joblib, Hugging Face Transformers, PyTorch, pdfplumber, SentencePiece
Database	MongoDB (local or MongoDB Atlas)
Tools	Visual Studio Code, Git and GitHub, Postman and Swagger UI, ESLint, Google Colab (YOLO training)

5.2 Component Implementation

5.2.1 Frontend assessment engine

Global session state is held in an AssessmentContext implemented with React's useReducer hook. It stores the student, lesson, questions per level, current level and question index, session identifier, collected responses, current expression, and the previous weak areas and score. The NEXT_QUESTION action moves to the next question of the current level; when the level is exhausted it moves to the next level, or ends the session after level 3, so every level is visited once and a session always contains nine questions. It also receives the decision made by the adaptive policy at the end of the level: when the last answer of the level was incorrect under high load, the next level starts in support mode (a supportMode flag in the state), which the AssessmentPage uses to open the hint at the start of each question. This applies after level 1 as well as after level 2. An earlier version jumped back to the first question of

a lower level on a scale-down, which repeated questions and made session length vary; this was replaced by the fixed-length design.

The AssessmentPage implements the per-question cycle. Selecting a different option increments the answer-change counter. A useTimer hook measures response time. On submission the page stops the timer, determines correctness, updates experience points and energy, requests the learning-state prediction, computes the cognitive load score and, for the last question of a level, the target level. It then evaluates the feedback policy and saves the response. All quiz decisions are implemented as pure functions in quizLogic.js (evaluateResponse, evaluateProactiveSupport, calculateCognitiveLoad, getAdaptiveNextLevel and shouldEnterSupportMode), which keeps the decision logic independent of the user interface and easy to test. Figure 5.1 shows the key functions.

```
export const calculateCognitiveLoad = ({ responseTime, answerChanges,
expression }) => {
  let score = 0;
  if (responseTime > 45) score += 40;          // overload zone
  else if (responseTime > 25) score += 20;     // elevated effort
  score += Math.min(answerChanges * 10, 30);   // metacognitive uncertainty
  if (expression === 'confused') score += 20;
  if (expression === 'frustrated') score += 30;
  if (expression === 'happy') score -= 10;     // positive affect
  return Math.max(0, Math.min(score, 100));
};

export const getAdaptiveNextLevel = (level, isCorrect, load) => {
  if (isCorrect && load < 35) return Math.min(level + 1, 3);        //
mastery
  if (!isCorrect && load > 65 && level > 1) return level - 1;       //
overload
  return level;                                                     //
productive struggle
};

export const shouldEnterSupportMode = (isCorrect, load) =>
  !isCorrect && load > 65;              // next level starts with hints
opened
```

Figure 5.1: Cognitive load, adaptive-level and support-mode functions (quizLogic.js, abridged)

The useWebcam hook requests a hidden 320×240 front camera stream, draws the current video frame onto an off-screen canvas at a fixed interval, encodes it as JPEG (quality 0.82) and posts it to the detect-emotion endpoint. To suppress noise from blinks and lighting changes, low-confidence results are discarded and the displayed expression changes only when at least two labels in a short rolling buffer agree (a majority vote). The stand-alone version sampled every 1.5 s, discarded results below

confidence 0.28 and used a five-label buffer; the merged version samples about every 800 ms, skips a tick while a request is still in flight, aborts requests after 4 s, discards results below 0.15 and uses a four-label buffer. On any error the last known expression is kept, and without a camera the label stays neutral so the quiz still works.

The gamification bar shows a mental-energy bar (HP, −15 for a wrong answer and +10 for a correct one), experience points (+25 for a correct answer) and a rank. An animated research avatar and research charts (an emotional-valence timeline and a per-question load chart) complete the learner interface.

Figures 5.2 to 5.4 show the learner interface. Before the quiz, the lesson-completed page shows the score of the previous attempt and the weak concepts that will be prioritised in the new session, together with the three levels of the assessment (Figure 5.2). During a question, the page shows the lesson and level, the timer, the answer-change counter, the gamification bar, the level path with progress out of nine questions and a small face-tracking panel with the current expression (Figure 5.3). After submission, the selected option is marked as correct or incorrect and the predicted learning state is shown above the question (Figure 5.4).

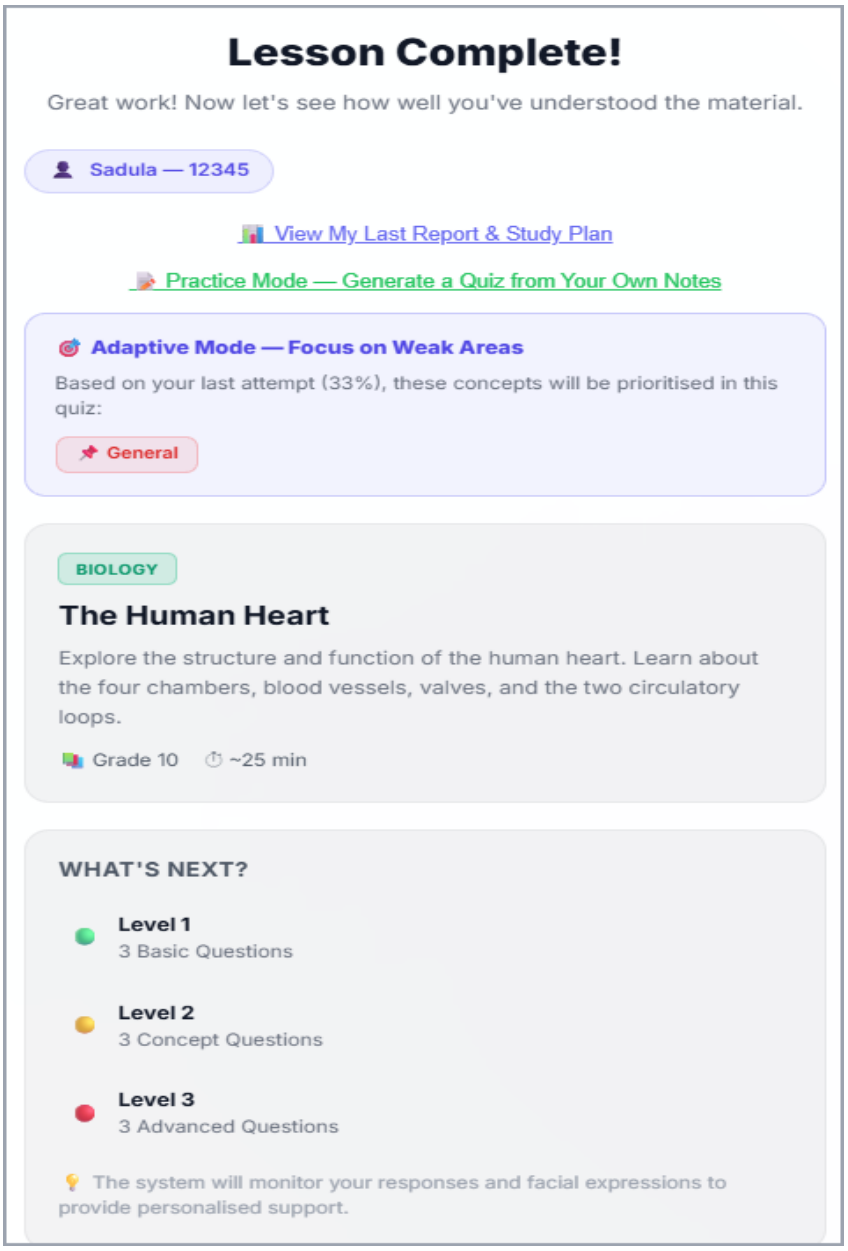


Figure 5.2: Lesson-completed page with the weak concepts prioritised in the next session

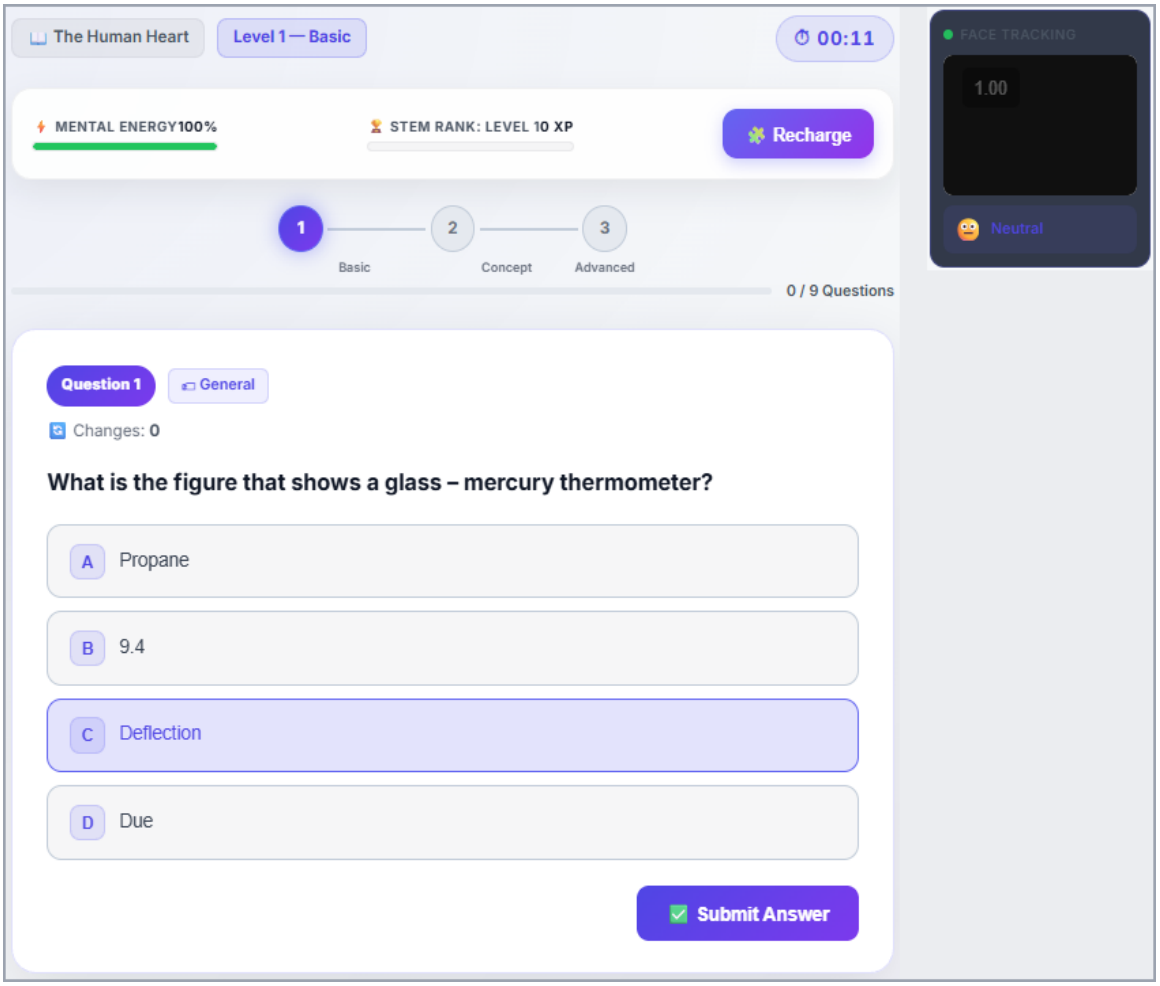


Figure 5.3: Assessment page during a Level 1 question, with the face-tracking panel

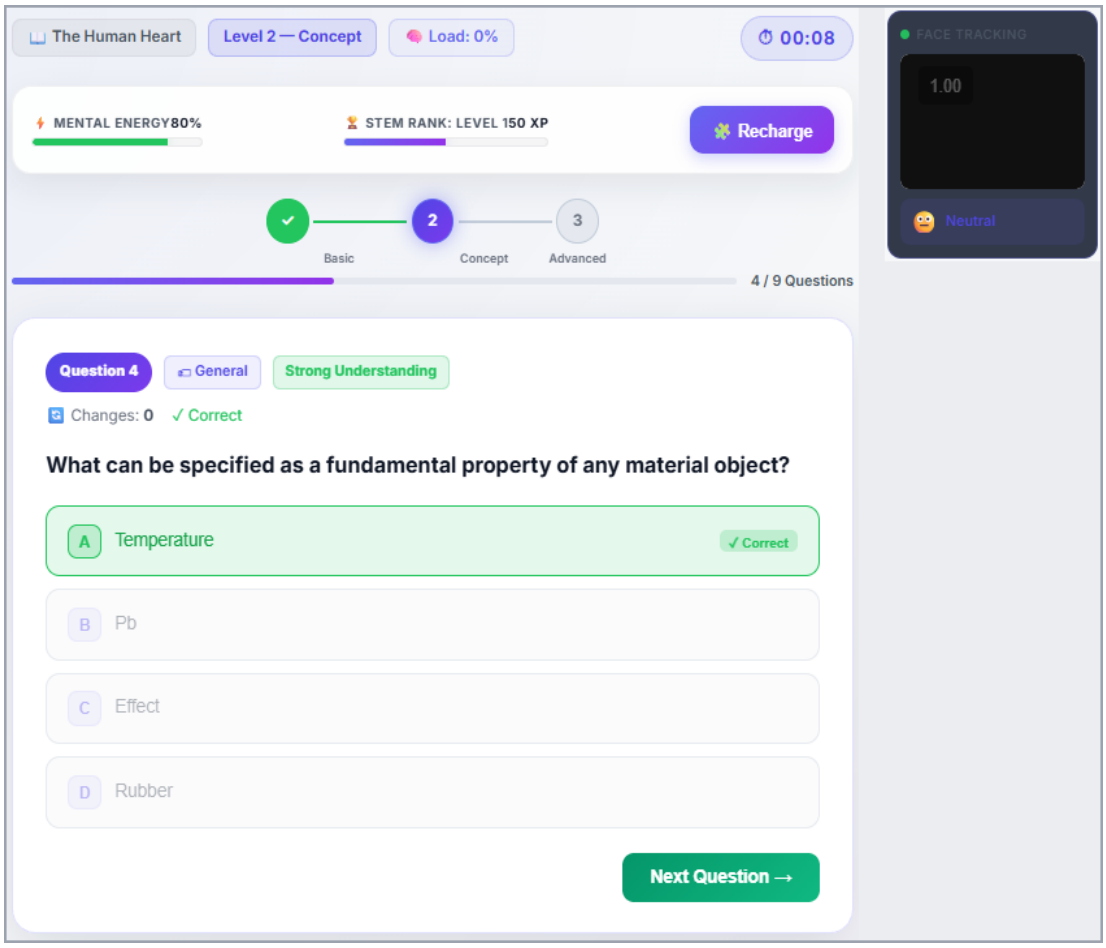


Figure 5.4: Feedback after a correct answer, with the predicted learning state

5.2.2 Backend API

The component exposes its endpoints through nine routers (Table 5.2); in the merged build all paths are prefixed with /api/adaptive-quiz instead of /api, and a /status endpoint reports readiness. At start-up it initialises MongoDB through Beanie and loads the YOLO model in a background thread so that the first webcam request is not delayed. A custom exception handler converts validation and HTTP errors into the {success, message} format expected by the frontend.

Table 5.2: Main backend endpoints

Endpoint	Access	Purpose
POST /api/auth/register, /login; GET /profile	Public / JWT	Account creation and login (10 requests per minute)
GET /api/lessons, /api/lessons/{id}	Public	Lesson catalogue
GET /api/assessments/{lesson}/adaptive/{student}	JWT, owner	Adaptive question set using previous weak areas
POST /api/responses, /responses/bulk	JWT, owner	Save responses

Endpoint	Access	Purpose
POST /api/predict-learning-state	JWT	Proxy to ML /predict with rule fallback
POST /api/detect-emotion	Public	YOLOv8 expression from a frame
GET /api/reports/{student}/{lesson}, /history, /progress	JWT, owner	Compute and store report; history; progress timeline
GET /api/reports/all	Teacher key	All reports for the teacher dashboard
POST /api/questions/generate	Teacher key	PDF to questions, saved to the bank
POST /api/questions/practice-generate	JWT	PDF to private practice questions (not saved)
GET, PUT, DELETE /api/study-plan/{lesson}	JWT	Saved customised study plan

5.2.3 ML service

The ML service exposes /predict, /generate-questions and /detect-emotion. The prediction module loads the joblib payload (model, label encoder and feature order) once and reuses it. The T5 generator is pre-loaded during start-up in a worker thread; downloads use a single worker with retries because parallel downloads were unreliable on some Windows machines. The training script (train.py) prints the label distribution, five-fold cross-validation accuracy and feature importances, retrains on all data, reports a stratified 80/20 hold-out evaluation and saves the model.

5.2.4 Teacher dashboard

The teacher dashboard (Figures 5.5 and 5.6) is protected by a teacher key entered once and sent as an X-Teacher-Key header. It loads all assessment reports, groups sessions by student and sorts students with the lowest average first. Each student card shows the latest and average score, the trend, a score-history strip, the dominant emotion, average hint use, weak-area chips, an expandable session list and a status of Mastery, Progressing or Needs Support. All reports can be exported as JSON for research analysis. The dashboard also contains the question-generator panel in which the teacher uploads a PDF, chooses a lesson, concept tag and number of questions, and reviews the generated questions grouped by level.

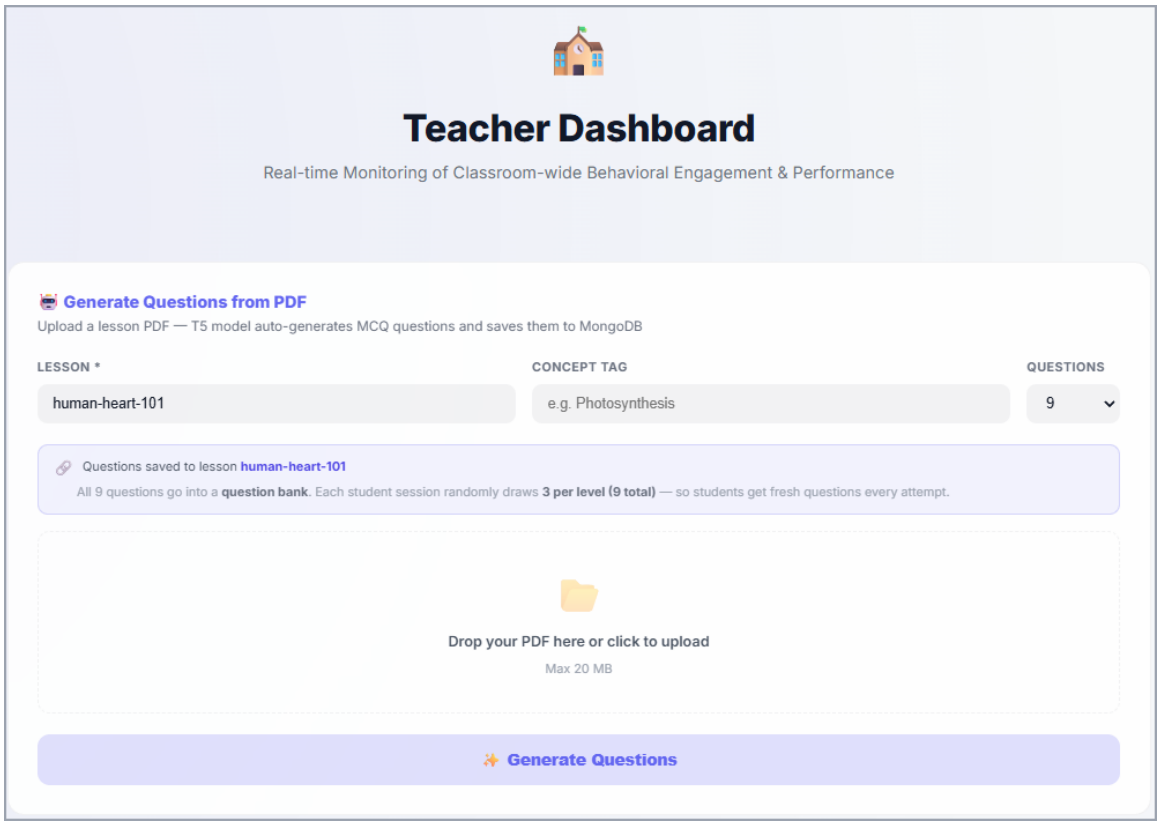


Figure 5.5: Teacher dashboard with the PDF question generator

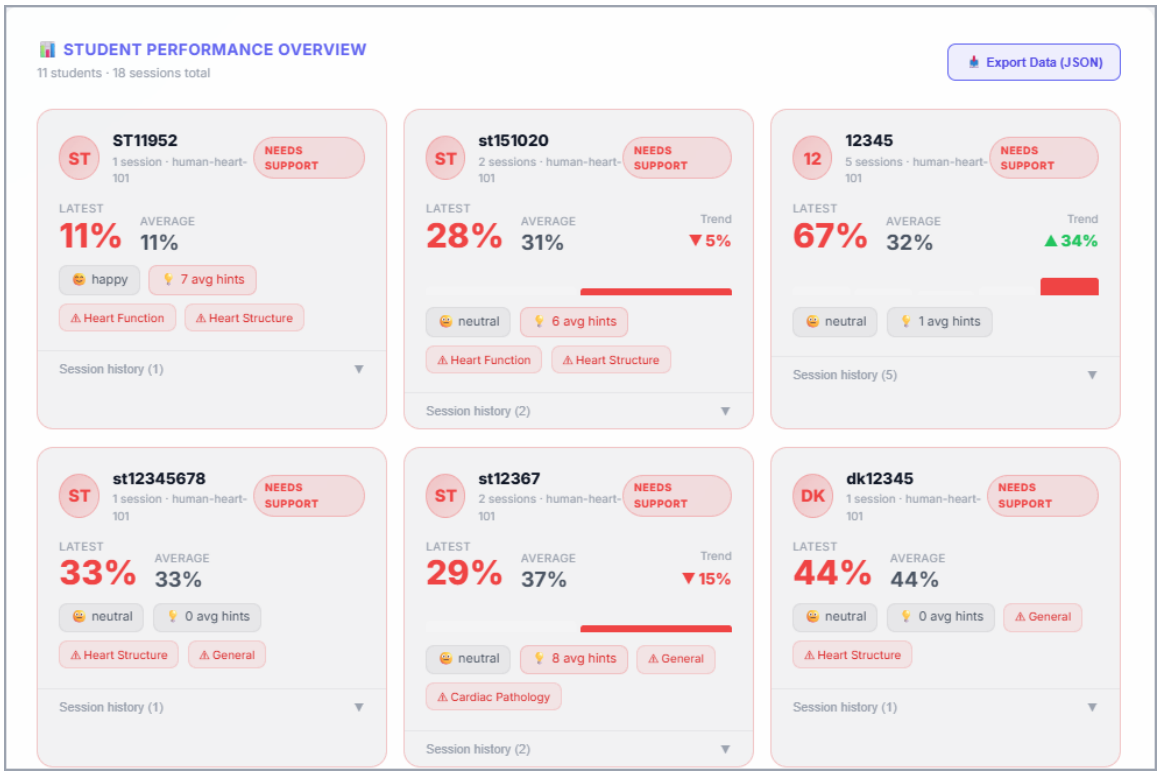


Figure 5.6: Student performance overview in the teacher dashboard

5.2.5 Personalised study plan and timetable

The study plan is built on the report page from the session report, using pure functions in reportUtils.js, so it can be regenerated at any time from stored data. It has two parts: a set of action cards (generateStudyPlan) and a seven-day timetable (generateWeeklyTimetable). Achievement badges (calculateBadges) are computed in the same way.

Action cards. One card is created for every weak concept. It states the learner's current score on that concept, sets a target of the current score plus 20 points (capped at 90%), asks the learner to complete at least four practice questions and retake the quiz, and gives a real calendar deadline seven days ahead (for example "You're at 33% on Heart Structure … aim for 53%+"). If there are no weak areas and the score is above 90%, an advanced challenge with a three-day deadline is suggested instead. Behavioural cards are added from the session data: an average load above 70 suggests splitting the next session into three 10-minute blocks with 5-minute breaks; a load below 30 with a score above 80% suggests moving to the next difficulty tier; more than five minutes of total time produces a persistence card; a dominant confused or frustrated expression suggests taking a short rest before studying; and three or more hints suggest re-reading the theory before using hints. A final card recommends the next action: revisiting the heart model and retaking Level 1 with a 70% target when the score is below 60%, or an advanced module otherwise. Every card carries a numeric metric and a deadline so that progress can be checked.

Weekly timetable. The timetable covers seven real calendar days starting from today. Its intensity is set from the total score (Table 5.3), and topics are taken from a prioritised queue: weak areas first (review, high priority), then concepts scored between 60% and 80% (practice, medium priority); when there are none, challenge topics such as Advanced Concepts and Problem Solving are used. On weekdays the main session is placed after school at 4:00 PM and at weekends in the morning at 8:00 AM. Medium and intensive plans add 15-minute evening flashcard sessions at 7:00 PM, and a weekly practice quiz is placed on the first weekend day in the afternoon (20 minutes, or 40 minutes for intensive plans). The total weekly study time is shown with the plan.

Table 5.3: Timetable intensity levels

Total score	Intensity	Study days per week	Main session	Extra sessions
85% or above	Light	4	25 min	Weekly practice quiz
60–84%	Medium	6	30 min	Flashcards on two days, weekly quiz
Below 60%	Intensive	7	40 min	Flashcards on five days, 40-minute weekly quiz

Customisation. The learner can adapt the generated plan in four ways:

- choose a study length of quick (0.6×), standard (1.0×) or deep (1.5×), which scales every session;
- tap a day to skip it, or in edit mode mark any day as a rest day;
- in edit mode, change the topic, type, time slot (morning, afternoon, after school or evening) and duration of any session, and add or remove sessions;
- save the plan, or reset it to the generated version.

Saving sends the schedule, intensity, weekly minutes and week label to PUT /api/study-plan/{lessonId}. One plan is stored per student and lesson, so the customised plan is shown again at the next login with a “Customized” badge; reset (DELETE) removes it and the generated plan is shown again. The endpoints use the identity in the student's token rather than a student identifier from the request, so a learner can read and change only their own plan. In the stand-alone build the load-based cards depend on defect D1 being fixed, because cognitive load is read back from the stored responses.

Figure 5.7 shows the top of the report page, with the badges, the progress summary and the level-wise scores, and Figure 5.8 shows the generated weekly timetable with the session-length options and the Customize Plan button.



Figure 5.7: Assessment report page with badges, progress summary and level-wise performance

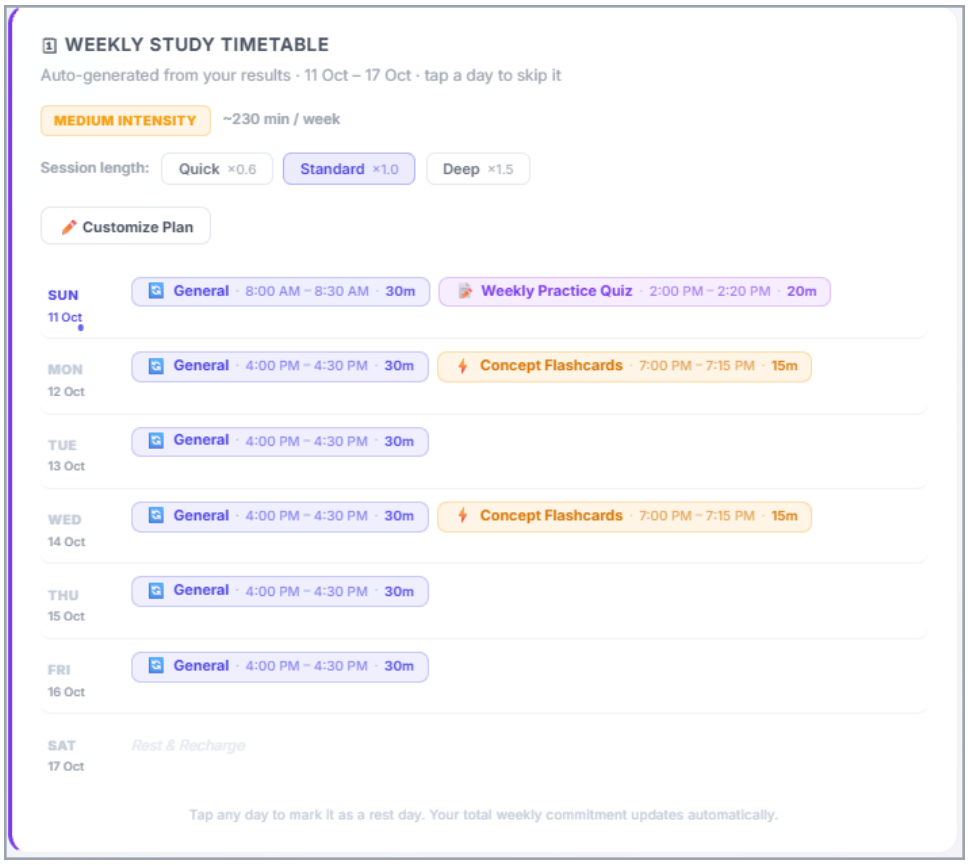


Figure 5.8: Generated weekly study timetable with customisation options

5.3 Integration with the Other Components

The component was integrated into Study Buddy STEM AI as a component package. Table 5.4 summarises the changes made during the merge. The route namespace moved from /api to /api/adaptive-quiz, and the frontend was rebuilt with the base path /adaptive-quiz and a matching router basename, so that the main application can serve the built files with a single-page-application fallback for deep links. Configuration was folded into the platform's shared settings module, and the MongoDB connection was moved into the shared application lifespan with failure isolation. Learning-state prediction gained the in-process fallback tier, and expression detection was fixed in the main application with single-flight inference so that a frame burst cannot stall prediction. The six data documents, the pure quiz and report logic, the ML service, the seed data and the JSON response format were carried over unchanged.

Table 5.4: Changes made when merging the component into Study Buddy STEM AI

Aspect	Stand-alone application	Merged into Study Buddy STEM AI
Processes	3 (frontend :5173, backend :3001, ML :8000)	2 (main app :8000 with API and built SPA; ML service :8001)
Routes	/api/*	/api/adaptive-quiz/*; frontend base URL and router basename updated
Backend code	Flat package backend/app/	Component package backend/components/adaptive_quiz/, mounted like the other components
Configuration and database	Own config.py and db.py; local MongoDB	Shared settings; guarded Mongo initialisation in the shared lifespan; MongoDB Atlas
Expression detection	Backend YOLO (ML service also had a ViT alternative)	Main app YOLO only, single-flight, dedicated executor, 256-pixel input
Learning-state prediction	ML service, then rules	ML service, then in-process Random Forest, then rules
Response data	cognitiveLoad and adaptiveLevel not declared (D1)	cognitiveLoad and adaptiveLevel declared on StudentResponse
Rate limiting	SlowAPI on login and registration	Removed with the app-wide middleware; to be re-added per route

Authentication uses the component's own student accounts and JWT tokens, and the lesson record's arModuleId links each assessment to the corresponding lesson. Within the integrated learning workflow, the assessment follows the storytelling and conversational components and precedes the Knowledge Maps component, so a learner can move from studying and questioning a concept to being assessed on it and then to consolidating it with their own notes.

5.4 Verification and Software Testing

Testing combined unit checks of the pure logic modules, automated API tests and model tests. The API tests used FastAPI's TestClient with the real routers, models and authentication, and an in-memory MongoDB substitute (mongomock-motor) seeded with the reference lesson; the ML service was deliberately left unreachable to exercise the fallback. Table 5.5 lists the executed test cases and their observed results.

Table 5.5: Executed test cases and observed results

ID	Test case	Observed result	Status
T1	Register with a 2-character student ID, invalid email and short password	HTTP 400 “Student ID must be at least 3 characters”	Pass
T2	Eleven failed logins within one minute	401 for attempts 1–10, 429 on attempt 11	Pass
T3	Request another student's history with a valid token	HTTP 403	Pass
T4	Teacher reports with no key, wrong key, correct key	403, 403, 200	Pass
T5	calculateCognitiveLoad(50 s, 4 changes, frustrated)	100	Pass
T6	calculateCognitiveLoad(8 s, 0 changes, happy)	0	Pass
T7	getAdaptiveNextLevel(1, correct, 20) and (3, correct, 10)	2 and 3 (capped)	Pass
T8	getAdaptiveNextLevel(2, incorrect, 80) and (1, incorrect, 90)	1 and 1 (floored)	Pass
T9	ML predict (correct, 8 s, 0, level 1, happy)	strong_understanding, p = 1.00	Pass
T10	ML predict (incorrect, 35.5 s, 3, level 2, confused)	needs_hint, p = 0.995	Pass
T11	Prediction with the ML service stopped (incorrect, frustrated)	needs_hint with fallback flag	Pass
T12	Blank grey frame sent to /detect-emotion	neutral, confidence 0.0, no error	Pass
T13	First session for a new student	isAdaptive false, 9 questions	Pass
T14	Upload a text file to the question generator	HTTP 400 “Only PDF files are allowed”	Pass
T15	Complete a 9-question session (5 correct, 4 hints, 20 s each) and open the report	Score 55.6%, level scores 100/33.3/33.3, but 18 stored responses, 8 hints, 360 s	Fail (D2)
T16	Check that cognitiveLoad is stored with responses	Field absent in stored responses	Fail (D1)
T17	Second session after four weak concepts	Before fix: 3 + 2 + 2 = 7 questions (D4). After fix: 3 + 3 + 3 = 9, one weak concept in each level	Pass after fix
T18	Generator on test PDF (T5 stubbed)	9 questions, 3 per level, 4 options each	Pass with quality issues (Section 6.2)
T19	2,000 simulated sessions with random scale-up, hold and scale-down decisions (reducer test)	Every session exactly 9 questions, no repeats; level after a scale-down in support mode	Pass
T20	Save, reload, update and reset a customised study plan; read it as another student	404 before saving; saved topic, duration and weekly minutes returned; update kept one plan; reset returned 404; other student 404	Pass
T21	evaluateResponse for a wrong answer (neutral and frustrated) and a correct answer	Wrong: explanation with correct answer shown; wrong and frustrated: hint and explanation; correct: neither	Pass

ID	Test case	Observed result	Status
T22	evaluateProactiveSupport with a frustrated expression at 9 s and 10 s, and neutral at 20 s	No message at 9 s; hint message at 10 s; no message for neutral at 20 s	Pass
T23	2,000 simulated sessions with random answers and load at each level end, using shouldEnterSupportMode (reducer test)	Every session exactly 9 unique questions; support mode started at level 2 in 347 and at level 3 in 322 sessions; 0 mismatches with the rule	Pass

The failed tests and code review identified four defects, one of which (D4) has since been fixed. They are reported openly because they affect the validity of the reported analytics:

- **D1. Cognitive load and adaptive level are not persisted (stand-alone build).** The frontend sends cognitiveLoad and adaptiveLevel with each response, but the stand-alone StudentResponse document does not define these fields, so they are dropped on save (T16). Report features that read cognitiveLoad from stored responses therefore see 0: the “Cool Under Pressure” badge is always awarded, the cognitive-overload study-plan advice never appears, and the load chart is flat. Both fields have since been added to the StudentResponse document in the merged Study Buddy STEM AI build; this should be confirmed with a repeat of T16 on that build.
- **D2. Responses are stored twice.** Each response is saved on submission and the full list is saved again through the bulk endpoint at the end of the session. In T15 the report aggregated 18 rows for a 9-question session, doubling the hint count (8 instead of 4) and total time (360 s instead of 180 s). Percentage scores are unaffected. The fix is to remove one of the two saves or to make saving idempotent on (sessionId, questionId).
- **D3. Two expression categories are unreachable.** The deployed detector produces only frustrated, neutral and happy, so the confused branches of the load score, hint policy and fallback rules are never triggered by real webcam input (Section 6.2).
- **D4. Adaptive sessions had a variable length (fixed).** When a level contained more than one weak-concept question, only the first was kept and the others were excluded from the random fill, so the reference bank produced 7 instead of 9 questions (T17). In addition, a scale-down re-opened the lower level, which

repeated questions and could stretch a session to 12 questions. Both were fixed: the adaptive endpoint now fills the remaining slots from all unused questions of the level, and the session follows a fixed 1–2–3 path in which a scale-down triggers support mode instead of a repeat (Section 4.4.4). T17 and T19 confirm that every session now has exactly nine questions.

5.5 Deployment and Operation

In the merged deployment the component runs inside the Study Buddy STEM AI application. On a fresh machine the platform dependencies and the component and ML requirements are installed, the .env file is configured (MONGO_URI for MongoDB Atlas, JWT_SECRET, TEACHER_KEY and ML_SERVICE_URL, which defaults to http://127.0.0.1:8001), the demo lesson and nine questions are seeded with python -m backend.components.adaptive_quiz.seed, and the React application is built with npm run build. The ML service is then started with uvicorn on port 8001 and the main application with python run.py, after which the quiz is available at http://127.0.0.1:8000/adaptive-quiz. The frontend must be rebuilt after any change to its source. The model files (best.pt and both .pkl files) are stored in the repository, and the T5 model is downloaded from Hugging Face on first start. If the ML service or the webcam is unavailable, the quiz still completes.

For production, the main application and the ML service can be containerised and deployed separately so that the ML service can be scaled or given a GPU independently. The JWT secret and teacher key must be replaced with strong random values, and CORS must be restricted to the production domain.

6.RESULTS AND VALIDATION

6.1 Research Results

6.1.1 Learning-state prediction (RQ1)

Table 6.1 and Figure 6.1 compare the proposed Random Forest with the baselines. The Random Forest achieved the highest cross-validated accuracy (97.9% ± 1.7%) and macro F1 (0.978). It outperformed the rule-based fallback by 10.0 percentage points and the majority-class baseline by 71.5 points, which supports the working hypothesis for RQ1. The simpler learned models, logistic regression and a decision tree of depth two, both reached 95.7%.

Table 6.1: Learning-state prediction, stratified five-fold cross-validation (n = 140)

Model	Accuracy (%)	SD (%)	Macro F1
Majority-class baseline	26.4	1.7	0.104
Rule-based fallback (no training)	87.9	–	0.880
Logistic regression	95.7	3.5	0.956
Decision tree (depth 2)	95.7	1.4	0.956
Random Forest (proposed)	97.9	1.7	0.978

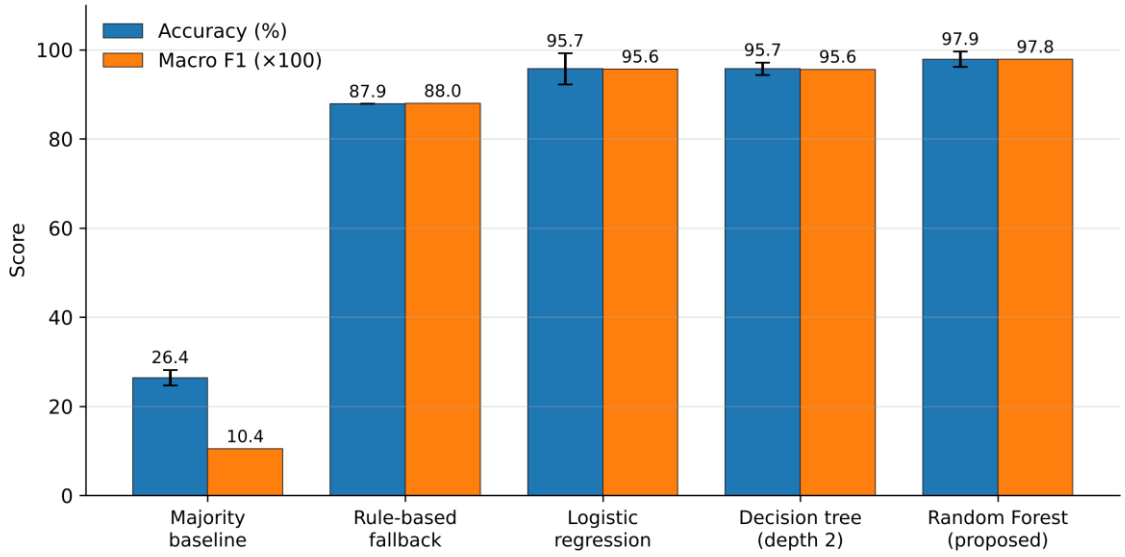


Figure 6.1: Cross-validated accuracy (with standard deviation) and macro F1 of the proposed model and baselines

Table 6.2 gives per-class results from the out-of-fold predictions, and Figure 6.2 shows the confusion matrix. Weak understanding and needs hint were classified without error. All three errors occurred between strong and partial understanding, the two states that both have correct answers and differ only in time and answer changes.

Table 6.2: Per-class results of the Random Forest (out-of-fold predictions)

Learning state	Precision	Recall	F1	Support
Strong understanding	0.938	0.968	0.952	31
Partial understanding	0.973	0.947	0.960	38
Weak understanding	1.000	1.000	1.000	38
Needs hint	1.000	1.000	1.000	33
Macro average	0.978	0.979	0.978	140

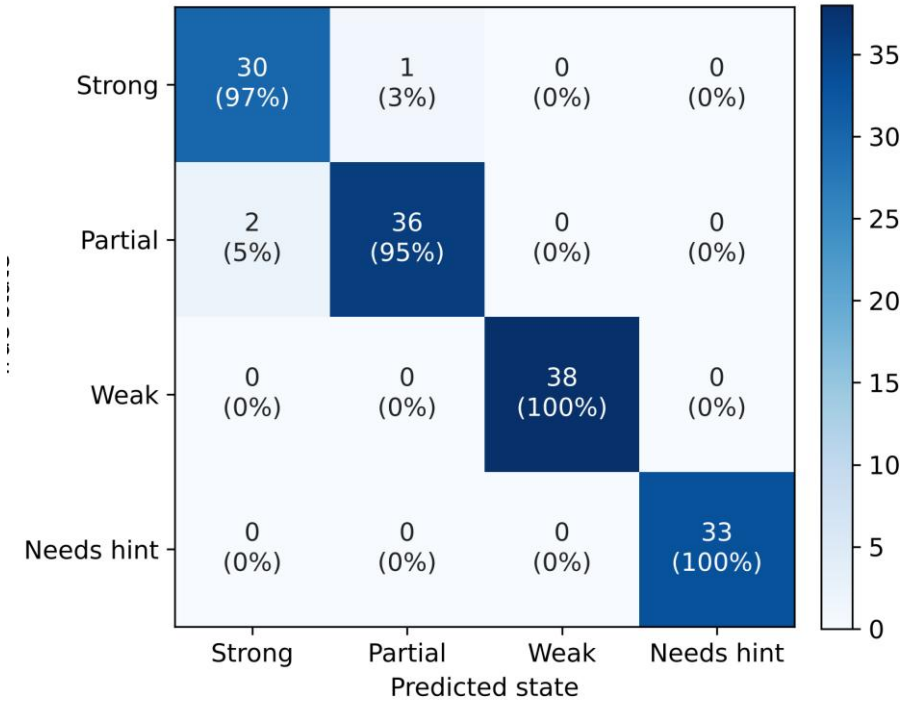


Figure 6.2: Confusion matrix of the Random Forest (out-of-fold predictions; counts and row percentages)

6.1.2 Feature contribution (RQ2)

Figure 6.3 shows the feature importances: correctness (0.381), response time (0.293), answer changes (0.202), facial expression (0.110) and quiz level (0.014). The ablation in Table 6.3 tells a stronger story. Removing correctness reduced accuracy to 63.6%. Removing response time or answer changes reduced it only slightly (to 97.1%), because each can partly substitute for the other. Removing facial expression or quiz level did not change accuracy at all.

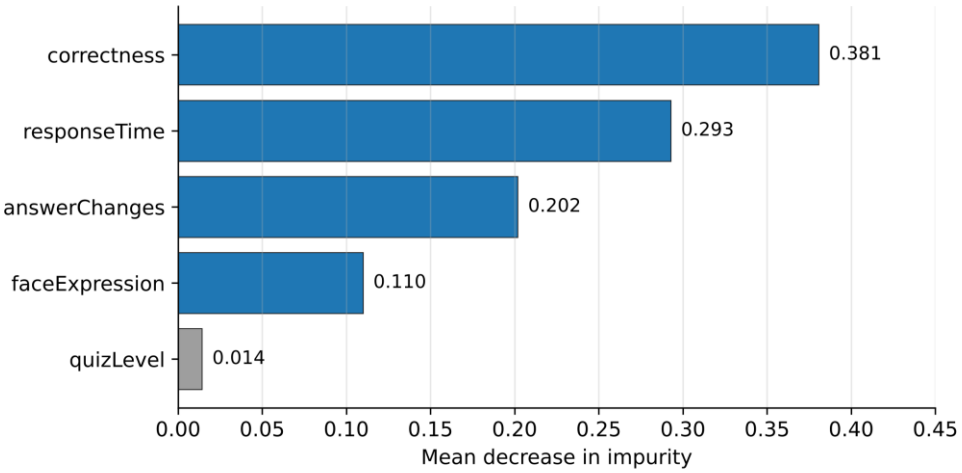


Figure 6.3: Random Forest feature importance (mean decrease in impurity)

Table 6.3: Leave-one-feature-out ablation of the Random Forest

Feature removed	Accuracy (%)	Change (points)
None (all features)	97.9	–
correctness	63.6	−34.3
responseTime	97.1	−0.7
answerChanges	97.1	−0.7
faceExpression	97.9	0.0
quizLevel	97.9	0.0

A further analysis explains these results. A decision tree limited to two levels learned the rule “if incorrect: needs hint when answer changes > 2.5, otherwise weak; if correct: strong when time ≤ 14.5 s, otherwise partial”, and this two-rule model alone reached 95.7%. The labels in the current dataset are therefore almost a deterministic function of correctness plus one behavioural feature, and the expression feature is largely redundant with answer changes and time. The high accuracy shows that the model learns the intended patterns, but it should not be read as evidence that facial expression improves real-world prediction.

6.1.3 Adaptive-policy behaviour (RQ4)

Table 6.4 and Figure 6.4 summarise 500 simulated sessions per learner profile. The policy behaved as designed. For high performers the mean load was 2.5 and 57.1% of level-end decisions were scale-ups (the remaining decisions were mostly at level 3, where scaling up is not possible), and none of their level transitions entered support mode. For struggling learners the mean load was 76.3, 74.4% of questions were above the overload threshold of 65, and 45.8% of the transitions to the next level (after level 1 or level 2) were made in support mode. Average learners stayed mostly in the productive-struggle zone, with 27.9% scale-ups and 2.0% of transitions in support mode.

Table 6.4: Simulated behaviour of the adaptive policy (500 sessions per profile)

Measure	High performer	Average learner	Struggling learner
Mean cognitive load (0–100)	2.5	28.8	76.3
Questions with load > 65 (%)	0.0	6.0	74.4
Level-end decisions: scale up (%)	57.1	27.9	0.6
Level transitions in support mode (%)	0.0	2.0	45.8
Most frequent predicted state	Strong (68%)	Partial (52%)	Needs hint (58%)

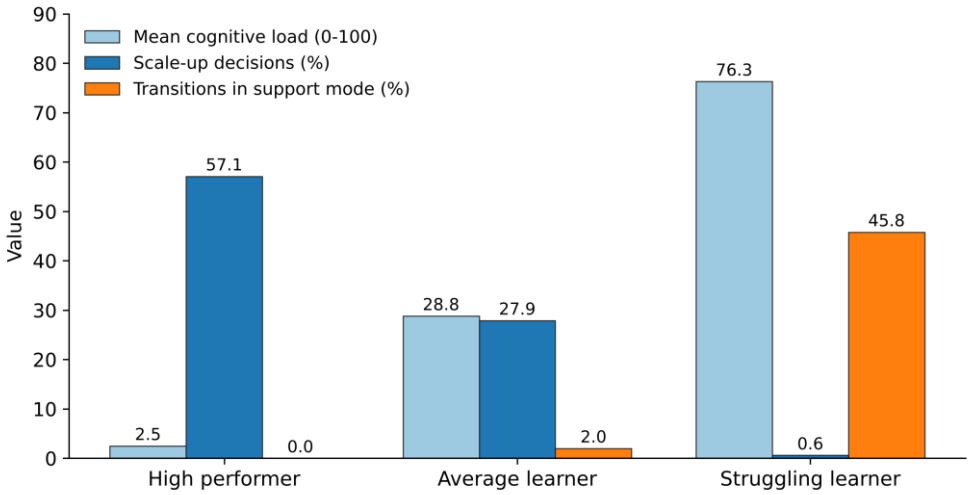


Figure 6.4: Mean cognitive load and adaptive decisions per simulated learner profile

6.2 Product and Technical Results

6.2.1 Facial-expression detector (RQ3)

The fine-tuned YOLOv8s detector reached a validation precision of 0.847, recall of 0.866, mAP@0.5 of 0.905 and mAP@0.5:0.95 of 0.702 (Table 6.5). Figure 6.5 shows that mAP rose quickly during the first ten epochs and levelled off after about epoch 40, with the best mAP@0.5:0.95 at epoch 56. The validation classification loss stopped improving after about epoch 40 while the training loss kept falling, especially in the last ten epochs, which indicates mild over-fitting at the end of training. The sharp drop in all three training losses at epoch 50 is expected: Ultralytics switches off mosaic augmentation for the last ten epochs (close_mosaic = 10), which makes the training images easier. On the CPU of the test machine, one frame took about 206 ms end to end, comfortably inside the 1.5-second sampling interval.

Table 6.5: Expression detector validation metrics (from the trained checkpoint)

Metric	Value
Classes	negative, neutral, positive
Base model / epochs / image size	YOLOv8s / 60 / 640
Precision	0.847
Recall	0.866
mAP@0.5	0.905
mAP@0.5:0.95	0.702
CPU inference per frame	≈ 206 ms

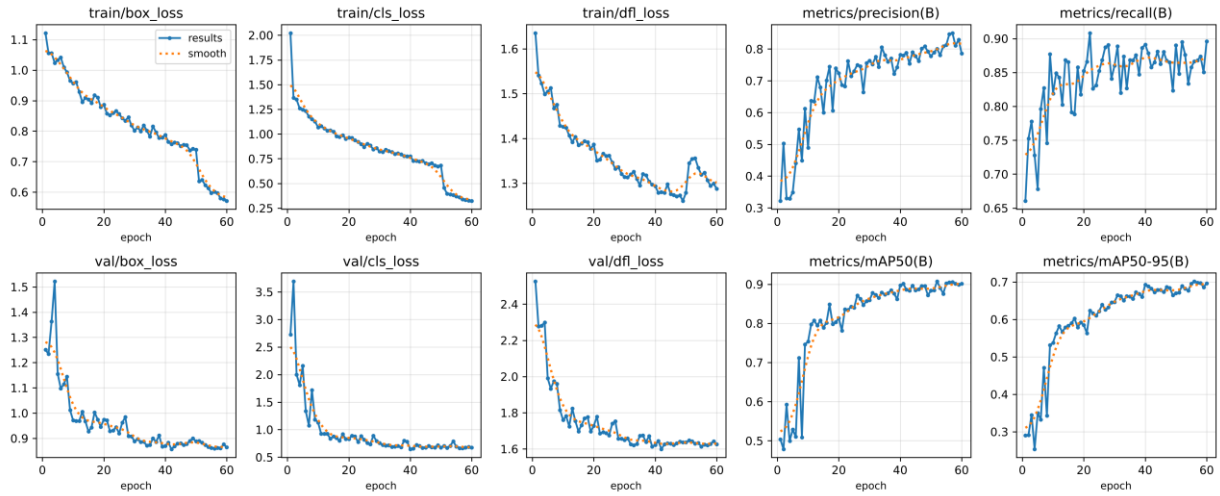


Figure 6.5: YOLOv8s training curves over 60 epochs: training and validation losses, precision, recall and mAP (blue: per-epoch values; orange: smoothed)

An important integration finding is that the detector has only three classes. After mapping, the system produces frustrated, neutral and happy, but never confused or surprised. The parts of the design that depend on confusion (the +20 load term, the 15-second confused hint and the confused condition in the fallback rules) are therefore inactive with the deployed model, and 33 of the 140 training records describe an expression that the live system never produces.

6.2.2 Other technical results

- **Prediction latency.** A single Random Forest prediction took about 11.6 ms on CPU, so prediction adds no noticeable delay to feedback.
- **Fallback.** With the ML service stopped, the backend returned rule-based states with a fallback flag, and the assessment continued normally (T11).
- **Question generation.** On the 20-sentence test PDF, the pipeline extracted 20 sentences, 28 answer candidates and a pool of 24 distractor phrases, and produced nine questions with three per level. Quality problems were observed in the answers and distractors: the article "The" appeared as a distractor, a candidate answer ended with a stray word ("1945 after"), and numeric answers such as "206 bones" were paired with names such as "Albert Einstein" and "Japan", which makes the correct option easy to spot. The T5 model itself could not be downloaded in the evaluation environment, so question wording was not evaluated.

6.3 User or Stakeholder Validation

A formal user study with learners and teachers was not completed within the project period, so no claims are made about learning gains, usability or acceptance. To support the next phase, a validation protocol was prepared (Appendix A). It proposes a within-subjects study in which GCE O/L students complete the Human Heart lesson and then one adaptive and one non-adaptive assessment in counterbalanced order. The planned measures are pre-test to post-test gain, a short NASA-TLX workload rating [11] to validate the cognitive load score against self-report, the System Usability Scale [35], teacher interviews on the reports and generated questions, and logged behavioural data, which will also provide real labelled data to retrain the learning-state model.

6.4 Integration Results

Within the stand-alone build, the services were integrated and exercised together through the API tests: authentication, adaptive question retrieval, response saving, prediction with fallback, report generation, progress tracking, adaptive re-selection in the next session, expression detection and question-generation validation all worked through the backend gateway (Table 5.5). These tests also exposed defects D1, D2 and D4 (D4 has since been fixed and re-tested), which are data-flow problems between the frontend and the backend rather than failures of individual modules. The component was then merged into Study Buddy STEM AI as described in Section 5.3 and runs against MongoDB Atlas, with a status endpoint that reports database and model readiness. Its isolated initialisation means that a database or ML failure degrades only the quiz routes and never the other platform components. One integration gap was noted during the merge: the question-generation routes depend on the questions router, ML_SERVICE_URL and the teacher-key dependency all being present, and when one is missing the PDF generator returns HTTP 404.

6.5 Summary of Findings

Table 6.6 summarises the findings for each research question.

Table 6.6: Summary of findings by research question

RQ	Finding
RQ1	Supported on the current dataset: the Random Forest (97.9%, macro F1 0.978) outperformed the rule-based fallback (87.9%) and the majority baseline (26.4%), but simple two-rule models came close (95.7%).
RQ2	Correctness is essential; time and answer changes are informative and partly interchangeable; facial expression and quiz level added no accuracy in ablation.
RQ3	Partly supported: the detector is accurate on its validation set (mAP@0.5 0.905) and fast enough (≈206 ms per frame), but it provides only three valence classes, so confusion cannot be detected.
RQ4	Supported in simulation: load and level decisions accelerated fluent learners and put struggling learners into support mode, separating the profiles in the intended direction.
RQ5	Supported at the functional level: tests confirmed one weak-concept question per level and exactly nine questions on retakes (T17, T19), correct-answer feedback after mistakes and the frustration hint (T21, T22), and saving and resetting of the customised study plan (T20). The effect of these features on learning still has to be measured with real learners.

7.DISCUSSION

7.1 Interpretation of Findings

The results show that the component works as an integrated system and that its decision logic behaves as designed, but they also show where the evidence is still weak. The learning-state model learned the labelled patterns almost perfectly and was fast enough for real-time use. Its errors were confined to the boundary between strong and partial understanding, which is exactly where the distinction depends on a time threshold rather than on correctness. This is a reasonable error profile for an educational system: confusing strong and partial understanding has mild consequences (the next level may or may not be delivered in support mode), whereas the model never confused learners who need help with learners who do not.

The ablation results are the most important finding for the research direction. They show that, in the current dataset, correctness plus either response time or answer changes is almost sufficient, and that facial expression adds no accuracy. There are two possible explanations: either the expression signal really is redundant with behaviour, or the dataset was constructed in a way that made it redundant. Because the dataset was prepared from expected patterns, the second explanation is more likely, and the question of whether webcam affect improves learning-state prediction remains open. It can only be answered with logged data from real learners whose state is labelled independently, for example by teacher observation or self-report.

The expression detector is technically accurate on its validation data and fast enough for continuous sampling. Its three-class output, however, is a mismatch with the five-emotion design used by the theory, the training data and the decision rules. Confusion is the learning-centred emotion with the strongest support in the literature [3], and it cannot currently be observed. The component therefore relies on frustration (negative valence) and on behavioural proxies to detect struggle, which is defensible but narrower than intended.

The simulation shows that the cognitive load score and adaptation thresholds create clearly different paths for different learners: fluent learners are accelerated, overloaded learners are scaffolded down into support mode, and average learners

mostly stay in place. This supports the internal consistency of the policy, but the simulation uses profiles defined by the researcher, so it does not show that the thresholds are optimal for real learners.

For RQ5, the functional tests show that the learner-facing support features work as designed in one integrated system: mistakes are always followed by the correct answer and a theory explanation, frustration triggers the hint, retakes focus on previously missed concepts without changing the session length, and the customised study plan is stored per learner and can be reset. These results show reliability, not educational benefit; whether the features improve learning is the main question for the planned user study.

7.2 Comparison with Literature Baselines and Existing Solutions

Compared with computerised adaptive testing [5] and knowledge tracing [6], [7], the component adapts on fewer interactions (three per level) and uses process signals that those models ignore, at the cost of the psychometric rigour of calibrated item difficulty. It adapts level by level instead of item by item, which is simpler but coarser. Compared with affect-aware tutors [8], it uses only an ordinary webcam and a single model and runs in a browser, which makes it far easier to deploy; the trade-off is a smaller label set and no physiological signals. Engagement recognition from faces has been shown to approach human agreement [19], and the detector's mAP@0.5 of 0.905 is consistent with the feasibility of such signals, although detection metrics on a curated dataset are not directly comparable with in-classroom engagement accuracy.

Against the rule-based fallback, which is typical of hand-crafted adaptive quiz logic, the learned model improved accuracy by 10 points while also providing class probabilities. The question generator follows the answer-aware T5 approach used in recent AQG work [26], [10]; the distractor problems observed in Section 6.2 match the weaknesses that Kurdi et al. identify as the main open problem in the field.

7.3 Research Contribution

- An explicit, literature-grounded per-question cognitive load score that combines response time, answer revisions and facial affect, and a bidirectional difficulty policy that maps mastery and overload onto ZPD-based level changes.
- A fusion of behavioural and affective features for four-class learning-state prediction, with an evaluation against rule-based and simple baselines and an ablation that quantifies the contribution of each signal.
- Evidence, from the ablation and the class mismatch, that the value of webcam affect in such systems must be established with independently labelled real-learner data, and a protocol for doing so.

7.4 Product Contribution

- A working web application that delivers adaptive, multi-level post-lesson assessment with real-time webcam expression detection, hints, theory explanations, customisable study plans and PDF-based question generation.
- A teacher dashboard with cross-student reports and PDF-to-question generation that extends the question bank without manual authoring, and a private practice mode for learners.
- Personalised reports, study plans, weekly timetables, badges and cross-session weak-area targeting.
- A resilient service architecture in which the assessment continues when the ML service or the camera is unavailable.

7.5 Limitations and Threats to Validity

Construct validity. The learning-state labels are defined by the researcher rather than measured independently, and the cognitive load score has not yet been validated against an established measure such as NASA-TLX. The label “frustrated” is a mapping of negative valence, which may also include sadness or concentration.

Internal validity. The training data was created from expected patterns, which makes the classification task easier than reality and explains the near-perfect scores. Four duplicate records slightly inflate cross-validation results. Defects D1 (resolved in the merged build) and D2 mean that some report values (hint counts, total time,

load-based advice and badges) were not correct in the evaluated version; D4, which affected session length, has been fixed.

External validity. All results come from offline analysis and simulation with one reference lesson. The expression detector was validated on its own dataset, not on webcam images of the target learners in classroom lighting; performance may differ across ages, skin tones and camera quality. The generated-question evaluation used a single short document and could not include the T5 output.

Statistical conclusion validity. With 140 records, differences of one or two percentage points between models correspond to one to three records and are within the fold-to-fold standard deviation.

7.6 Implications

For practitioners, the results suggest that most of the value of behaviour-aware assessment can be obtained cheaply from correctness, time and answer revisions, which require no camera and no special hardware. Webcam affect should be treated as an optional, privacy-sensitive enhancement whose benefit has to be demonstrated. For researchers, the work shows the importance of ablation and simple baselines when reporting high accuracy on small educational datasets. For the product, the immediate implications are to fix the four data-flow defects, to align the expression classes with the design (either by training a detector with a confusion class or by redesigning the rules around valence), and to begin collecting real learner data.

8.COMMERCIALISATION AND SUSTAINABILITY

This chapter is not present in the body of the template supplied but is listed in its table of contents and is required by the SLIIT dissertation guidelines ("Commercialization aspects of the product").

8.1 Value Proposition and Target Users

The component turns any lesson into an adaptive assessment that notices when a learner is struggling and responds immediately, and it gives teachers a fast way to create leveled questions from their own material. The primary users are Sri Lankan GCE O/L students studying STEM subjects and their teachers. Secondary users are tuition institutes and e-learning providers in Sri Lanka and the region, where large class sizes make individual attention difficult, and parents who want insight into their children's progress.

8.2 Market and Competitive Position

The market includes engagement-focused quiz platforms, mastery-based adaptive learning platforms and learning management systems with quiz plug-ins. The component is differentiated by its combination of behaviour-aware support, explicit cognitive load monitoring, automatic question generation from teacher documents, and integration with lessons from the same platform. Its main competitive weaknesses are the lack of large-scale validation and the privacy concerns associated with webcams, which larger competitors avoid by not using cameras.

8.3 Business or Sustainability Model

A freemium model is proposed. Individual learners can use the core adaptive assessment free of charge, which supports adoption and the collection of consented learning data. Schools and tuition institutes pay a per-student annual subscription for the teacher dashboard, question generation, class analytics and integration with their lesson content. Because all models run on CPU and do not depend on paid external AI APIs, the marginal cost per assessment is low and mainly consists of hosting and database costs. A camera-free mode keeps the product usable in environments where webcams are unavailable or not permitted.

8.4 Scalability Intellectual Property and Compliance

- **Scalability.** The frontend is static and can be served from a CDN. The backend is stateless apart from the database and can be scaled horizontally. Expression inference is the heaviest per-user load (one frame every 1.5 seconds); it can be moved to a GPU service or to in-browser inference with a smaller exported model.
- **Intellectual property.** The cognitive load model, adaptation policy, reports and integration are original work. Ultralytics YOLOv8 is licensed under AGPL-3.0, which requires either releasing the service source code or purchasing an enterprise licence for closed commercial use. The T5 model and the other libraries use permissive licences.
- **Compliance.** Operation in Sri Lanka requires compliance with the Personal Data Protection Act No. 9 of 2022 [32], including parental consent for minors, purpose limitation and data security. Not storing images, and storing only expression labels, greatly reduces the compliance burden.

8.5 Contribution to the Sustainable Development Goals

The component contributes mainly to Sustainable Development Goal 4, Quality Education [36], by providing individualised feedback and support at low cost, which helps learners who receive little individual attention in large classes. Running on ordinary laptops without paid cloud AI services supports access in lower-resource schools (SDG 10, Reduced Inequalities).

9.CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This thesis presented a behaviour-aware, multi-level adaptive assessment component for a STEM learning platform. The component observes how a learner answers, not only what they answer, by combining correctness, response time, answer revisions and webcam-based facial expression. It predicts a learning state with a Random Forest, computes a literature-grounded cognitive load score, provides proactive and post-answer support, adapts difficulty in both directions across three levels, and personalises later sessions through weak-area tracking, reports and study plans. Teachers can extend the question bank automatically from PDF material.

The evaluation showed that the learning-state model outperformed rule-based and trivial baselines (97.9% cross-validated accuracy), that the expression detector is accurate and fast enough for continuous use, and that the adaptive policy separates learner profiles as intended. It also showed, honestly, that the current dataset is nearly rule-generated, that facial expression did not improve prediction on it, that the deployed detector cannot recognise confusion, and that four data-flow defects affected some analytics. The component is therefore a complete and working prototype whose research claims about affect now need to be tested with real learners.

9.2 Achievement of Objectives

Table 9.1 shows how far each research objective was achieved and where the supporting evidence is reported.

Table 9.1: Achievement of the research objectives

Objective	Status	Evidence
1. Literature review and feature derivation	Achieved	Chapter 2; thresholds in Section 4.4
2. Learning-state model	Achieved (on prepared data)	Section 6.1.1; Tables 6.1–6.3
3. Real-time expression detection and frustration hint	Partly achieved	Section 6.2.1; three classes only
4. Cognitive load, adaptation and wrong-answer feedback	Achieved	Sections 4.4.3–4.4.5, 6.1.3
5. PDF question generation	Achieved; quality needs work	Sections 4.4.9, 6.2.2
6. Reports, customisable study plan and retake prioritisation	Achieved; D4 fixed, D2 to fix	Sections 4.4.8, 5.4
7. Verification and evaluation	Achieved offline; user study pending	Chapters 5 and 6; Appendix A

9.3 Individual Research and Product Contributions

The individual research contribution is the design and evaluation of a fused behavioural and affective approach to learning-state prediction and cognitive load estimation that directly drives difficulty adaptation, together with an analysis of the contribution of each signal. The individual product contribution is the complete adaptive assessment component described in Section 1.8, from the learner and teacher interfaces to the backend, models and evaluation scripts.

9.4 Future Directions

1. Fix the remaining defects: confirm the D1 fix on the merged build, remove duplicate saving (D2), align the expression classes with the decision rules (D3), and re-add rate limiting to the authentication routes.
2. Run the planned user study (Appendix A) with ethical approval, collect real behavioural data with independent labels, and retrain and re-evaluate the learning-state model, including a fair test of whether facial expression adds value.
3. Train or adopt an expression model that includes a confusion class, and evaluate it on webcam images of the target age group under classroom conditions, including fairness across demographic groups.
4. Validate the cognitive load score against NASA-TLX and tune the thresholds from data.
5. Replace level-by-level adaptation with item-level selection using calibrated difficulty, for example through item response theory or knowledge tracing combined with the behavioural features.
6. Improve question generation with type-aware distractors (numbers with numbers, people with people), stop-word filtering, and teacher review before questions enter the graded bank.
7. Move expression inference into the browser to reduce latency and to keep camera frames on the learner's device.

REFERENCES

[1] J. Sweller, J. J. G. van Merriënboer, and F. Paas, "Cognitive architecture and instructional design: 20 years later," Educational Psychology Review, vol. 31, no. 2, pp. 261–292, 2019.

[2] F. Paas and J. J. G. van Merriënboer, "Cognitive-load theory: Methods to manage working memory load in the learning of complex tasks," Current Directions in Psychological Science, vol. 29, no. 4, pp. 394–398, 2020.

[3] J. M. Lodge, G. Kennedy, L. Lockyer, A. Arguel, and M. Pachman, "Understanding difficulties and resulting confusion in learning: An integrative review," Frontiers in Education, vol. 3, art. no. 49, 2018.

[4] A. Arguel, L. Lockyer, G. Kennedy, J. M. Lodge, and M. Pachman, "Seeking optimal confusion: A review on epistemic emotion management in interactive digital learning environments," Interactive Learning Environments, vol. 27, no. 2, pp. 200–210, 2019.

[5] K. T. Han, "Components of the item selection algorithm in computerized adaptive testing," Journal of Educational Evaluation for Health Professions, vol. 15, art. no. 7, 2018.

[6] G. Abdelrahman, Q. Wang, and B. Nunes, "Knowledge tracing: A survey," ACM Computing Surveys, vol. 55, no. 11, art. no. 224, 2023.

[7] S. Shen, Q. Liu, Z. Huang, Y. Zheng, M. Yin, M. Wang, and E. Chen, "A survey of knowledge tracing: Models, variants, and applications," IEEE Transactions on Learning Technologies, vol. 17, pp. 1858–1879, 2024.

[8] P. J. Standen et al., "An evaluation of an adaptive learning system based on multimodal affect recognition for learners with intellectual disabilities," British Journal of Educational Technology, vol. 51, no. 5, pp. 1748–1765, 2020.

[9] B. Wisniewski, K. Zierer, and J. Hattie, "The power of feedback revisited: A meta-analysis of educational feedback research," Frontiers in Psychology, vol. 10, art. no. 3087, 2020.

[10] G. Kurdi, J. Leo, B. Parsia, U. Sattler, and S. Al-Emari, "A systematic review of automatic question generation for educational purposes," International Journal of Artificial Intelligence in Education, vol. 30, no. 1, pp. 121–204, 2020.

[11] S. Said, M. Gozdzik, T. R. Roche, J. Braun, J. Rössler, A. Kaserer, D. R. Spahn, C. B. Nöthiger, and D. W. Tscholl, "Validation of the raw National Aeronautics and Space Administration Task Load Index (NASA-TLX)

questionnaire to assess perceived workload in patient monitoring tasks: Pooled analysis study using mixed models," Journal of Medical Internet Research, vol. 22, no. 9, art. no. e19472, 2020.

[12] P. H. Winne, "Theorizing and researching levels of processing in self-regulated learning," British Journal of Educational Psychology, vol. 88, no. 1, pp. 9–20, 2018.

[13] J. Camacho-Morles, G. R. Slemp, R. Pekrun, K. Loderer, H. Hou, and L. G. Oades, "Activity achievement emotions and academic performance: A meta-analysis," Educational Psychology Review, vol. 33, no. 3, pp. 1051–1095, 2021.

[14] A. A. Margolis, "Zone of proximal development, scaffolding and teaching practice," Cultural-Historical Psychology, vol. 16, no. 3, pp. 15–26, 2020.

[15] A. Nickow, P. Oreopoulos, and V. Quan, "The impressive effects of tutoring on PreK-12 learning: A systematic review and meta-analysis of the experimental evidence," National Bureau of Economic Research, Cambridge, MA, USA, Working Paper 27476, 2020.

[16] W. Oliveira, I. I. Bittencourt, S. Isotani, D. Dermeval, L. B. Marques, and I. F. Silveira, "Flow theory to promote learning in educational systems: Is it really relevant?," Revista Brasileira de Informática na Educação, vol. 26, no. 2, pp. 29–59, 2018.

[17] E. S. Mousavinasab, N. Zarifsanaiey, S. R. Niakan Kalhori, M. Rakhshan, L. Keikha, and M. Ghazi Saeedi, "Intelligent tutoring systems: A systematic review of characteristics, applications, and evaluation methods," Interactive Learning Environments, vol. 29, no. 1, pp. 142–163, 2021.

[18] Z. Xu, K. Wijekumar, G. Ramirez, X. Hu, and R. Irey, "The effectiveness of intelligent tutoring systems on K-12 students' reading comprehension: A meta-analysis," British Journal of Educational Technology, vol. 50, no. 6, pp. 3119–3137, 2019.

[19] M. A. A. Dewan, M. Murshed, and F. Lin, "Engagement detection in online learning: A review," Smart Learning Environments, vol. 6, art. no. 1, 2019.

[20] S. Li and W. Deng, "Deep facial expression recognition: A survey," IEEE Transactions on Affective Computing, vol. 13, no. 3, pp. 1195–1215, 2022.

[21] L. F. Barrett, R. Adolphs, S. Marsella, A. M. Martinez, and S. D. Pollak, "Emotional expressions reconsidered: Challenges to inferring emotion from human facial movements," Psychological Science in the Public Interest, vol. 20, no. 1, pp. 1–68, 2019.

[22] J. Terven, D.-M. Córdova-Esparza, and J.-A. Romero-González, "A comprehensive review of YOLO architectures in computer vision: From

YOLOv1 to YOLOv8 and YOLO-NAS," Machine Learning and Knowledge Extraction, vol. 5, no. 4, pp. 1680–1716, 2023.

[23] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," version 8.0.0, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[24] Z.-Q. Zhao, P. Zheng, S.-T. Xu, and X. Wu, "Object detection with deep learning: A review," IEEE Transactions on Neural Networks and Learning Systems, vol. 30, no. 11, pp. 3212–3232, 2019.

[25] R. Zhang, J. Guo, L. Chen, Y. Fan, and X. Cheng, "A review on question generation from natural language text," ACM Transactions on Information Systems, vol. 40, no. 1, pp. 1–43, 2021.

[26] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," Journal of Machine Learning Research, vol. 21, no. 140, pp. 1–67, 2020.

[27] J. Koivisto and J. Hamari, "The rise of motivational information systems: A review of gamification research," International Journal of Information Management, vol. 45, pp. 191–210, 2019.

[28] M. Sailer and L. Homner, "The gamification of learning: A meta-analysis," Educational Psychology Review, vol. 32, no. 1, pp. 77–112, 2020.

[29] scikit-learn developers, "scikit-learn: Machine learning in Python," 2025. [Online]. Available: https://scikit-learn.org

[30] A. Vabalas, E. Gowen, E. Poliakoff, and A. J. Casson, "Machine learning algorithm validation with a limited sample size," PLoS ONE, vol. 14, no. 11, art. no. e0224365, 2019.

[31] OWASP Foundation, "OWASP Top 10: 2021," 2021. [Online]. Available: https://owasp.org/Top10/

[32] Parliament of the Democratic Socialist Republic of Sri Lanka, Personal Data Protection Act, No. 9 of 2022, 2022.

[33] European Data Protection Board, "Guidelines 05/2020 on consent under Regulation 2016/679," version 1.1, May 2020. [Online]. Available: https://www.edpb.europa.eu

[34] M. Schonlau and R. Y. Zou, "The random forest algorithm for statistical learning," The Stata Journal, vol. 20, no. 1, pp. 3–29, 2020.

[35] J. R. Lewis, "The System Usability Scale: Past, present, and future," International Journal of Human–Computer Interaction, vol. 34, no. 7, pp. 577–590, 2018.

[36] United Nations Department of Economic and Social Affairs, The Sustainable Development Goals Report 2023: Special Edition. New York, NY, USA: United Nations, 2023.

GLOSSARY

Ablation – an experiment in which one part of a model, such as a feature, is removed to measure its contribution.

Answer change – one switch of the selected option to a different option before the answer is submitted.

Cognitive load score – the 0–100 per-question score defined in Section 4.4.3.

Learning state – the predicted category of a learner's understanding on one question: strong, partial or weak understanding, or needs hint.

Macro F1 score – the unweighted average of the F1 scores of all classes.

Productive struggle – effortful work at an appropriate level of challenge that leads to learning.

Weak area – a concept tag on which the learner scored below 60% in a session.

APPENDICES

Appendix A Research Instruments or Extended Results

A.1 Planned user-validation protocol

1. Obtain ethical clearance and informed consent (parental consent for minors).
2. Participants: at least 30 learners of the target grade and 3–5 STEM teachers.
3. Pre-test: a 10-question paper test on the Human Heart lesson.
4. Learners complete the lesson, then the adaptive assessment and a non-adaptive version (same question bank, fixed order, no hints), in counterbalanced order.
5. After each assessment: a 6-item NASA-TLX rating and a single-item confusion and frustration self-report.
6. Post-test equivalent to the pre-test, followed by the 10-item System Usability Scale.
7. Teachers review the reports and rate 20 generated questions for correctness, clarity and distractor quality on a 5-point scale.
8. Analysis: paired t-test or Wilcoxon test on learning gains and workload; correlation between the cognitive load score and NASA-TLX; retraining of the learning-state model on the logged data with independent labels.

A.2 System Usability Scale items

1. I think that I would like to use this system frequently.
2. I found the system unnecessarily complex.
3. I thought the system was easy to use.
4. I think that I would need the support of a technical person to be able to use this system.
5. I found the various functions in this system were well integrated.
6. I thought there was too much inconsistency in this system.
7. I would imagine that most people would learn to use this system very quickly.
8. I found the system very cumbersome to use.
9. I felt very confident using the system.

10. I needed to learn a lot of things before I could get going with this system.

A.3 Sample records from the learning-state dataset

Table A.1 shows example records from the learning-state training dataset described in Section 3.2.1.

Table A.1: Example records from training_data.csv

correctness	responseTime	answerChanges	quizLevel	faceExpression	label
1	8	0	1	happy	strong_understanding
1	10	0	1	neutral	strong_understanding
1	12	1	1	neutral	strong_understanding
0	40	4	3	frustrated	needs_hint

A.4 Application screenshots

Figures A.1 to A.3 show further screens of the component.

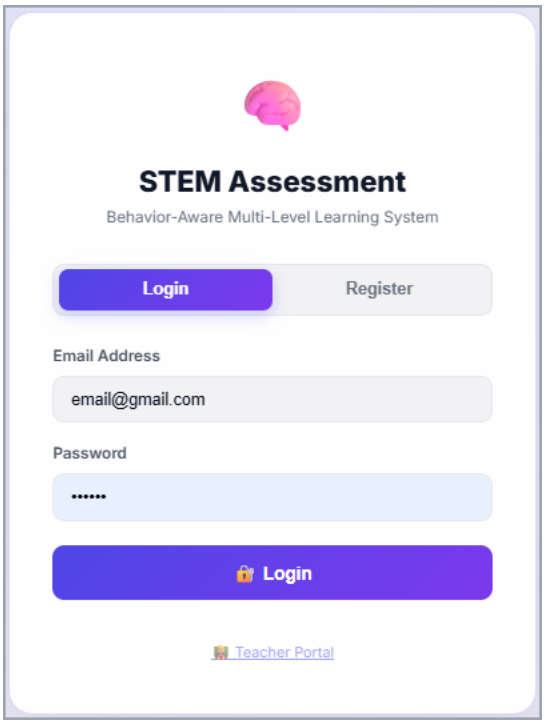


Figure A.1: Login and registration page

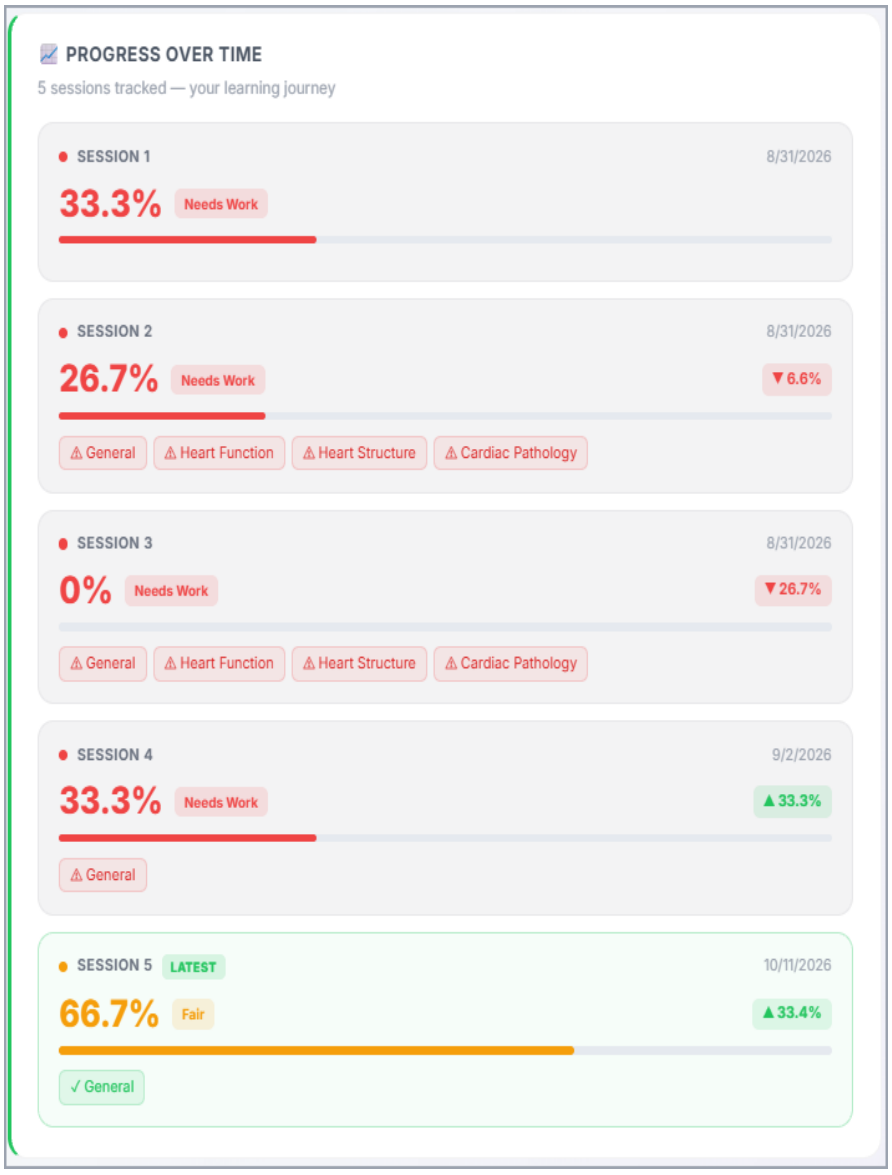


Figure A.2: Progress over time across five sessions of one learner



Figure A.3: Engagement timeline, cognitive effort chart and learning-state distribution in the report

Appendix B Academic Integrity and AI Use Disclosure

Assistance with reviewing code and improving the wording of selected sections of this thesis. The research, system design, implementation, testing, and analysis were carried out by the author. The author reviewed the final content and takes responsibility for the accuracy and originality of the work.

Table B.1 lists the generative AI and automated tools used in this work.

Table B.1: Use of generative AI and automated tools

Tool	Purpose	Parts affected
Claude (Anthropic)	Limited assistance with code review and wording improvements.	Selected thesis sections.
Python, scikit-learn, Ultralytics, LibreOffice	Implementing and testing the system and analysing experimental results.	Chapters 5 and 6